

EDGE-SECURE LAB - PHASE 5

Hybrid Overlay Networking, Centralized Logging, Intrusion Prevention, Disaster Recovery, and Scheduled Automation

Developed by Edinen Udofia

Table of Contents

EdgeSecure Hybrid Infrastructure Lab.....	6
PHASE 5: Hybrid Overlay Networking, Centralized Logging, Intrusion Prevention, Disaster Recovery, and Scheduled Automation.....	6
1. Introduction.....	6
2. Why the Original Plan Breaks: A First-Principles Look at Why Azure Cannot Reach 192.168.56.0/24.....	6
2.1 The assumption that needed correcting.....	6
2.2 What a Host-only adapter actually is.....	6
2.3 RFC 1918 and why this isn't just a VirtualBox quirk.....	6
2.4 Tracing the packet's actual death.....	7
2.5 Why NAT alone doesn't fix this either.....	7
2.6 The corrected design.....	7
3. Updated Lab Architecture.....	8
Part 1 — WireGuard: Hybrid Overlay Networking.....	10
1.1 Engineering Problem.....	10
1.2 First Principles.....	10
1.3 Architecture.....	11
1.4 Linux Internals.....	11
Why the kernel, not user space.....	11
The cryptographic primitives, and why each was chosen.....	12
Cryptokey Routing: the core abstraction.....	12
Why `PersistentKeepalive` matters here specifically.....	13
Kernel IP forwarding.....	13
1.5 Implementation.....	13
Step 1 — Package installation (`edge01`).....	13
Step 2 — Key generation (`edge01` and `backend02`, each independently).....	13
Step 3 — Listener configuration (`backend02`).....	14
Step 4 — Initiator configuration (`edge01`).....	14
Step 5 — IP forwarding and firewall zone (`edge01`).....	15
Step 6 — Bring the interface up (both nodes).....	15
1.6 Command Breakdown.....	15
1.7 Configuration Breakdown.....	16
1.8 Packet Flow.....	16
1.9 Process Flow.....	17
1.10 Verification.....	17
1.11 Failure Simulation.....	18
1.12 Troubleshooting.....	19
1.13 Production Engineering Notes.....	19
1.14 Interview Reference.....	20
Part 2 — rsyslog: Centralized Logging.....	21

2.1 Engineering Problem.....	21
2.2 First Principles.....	21
journald vs rsyslog — two different jobs, not competing tools.....	21
`/dev/log` and the Unix domain socket model.....	21
Why TCP, deliberately, over UDP.....	22
Why port 514.....	22
Why `/var/log/remote/` must exist as a structured hierarchy, not just a flat dump.	22
2.3 Architecture.....	23
2.4 Linux Internals.....	23
Reliable delivery and disk-assisted queues.....	23
2.5 Implementation.....	23
Step 1 — Server configuration (`edge01`).....	23
Step 2 — Client configuration (`backend01`).....	24
Step 3 — Client configuration (`backend02`).....	24
2.6 Command Breakdown.....	24
2.7 Configuration Breakdown.....	25
2.8 Packet Flow.....	25
2.9 Process Flow.....	26
2.10 Verification.....	26
2.11 Failure Simulation.....	26
2.12 Troubleshooting.....	27
2.13 Production Engineering Notes.....	27
2.14 Interview Reference.....	27
Part 3 — Fail2Ban: Automated Intrusion Response.....	29
3.1 Engineering Problem.....	29
3.2 First Principles.....	29
Filters and regex: turning unstructured text into structured events.....	29
Jails: binding a filter to an action and a threshold.....	29
The backend: how Fail2Ban actually watches a log file.....	29
How an enforcement decision actually becomes a network-level block.....	30
Where the block actually happens in the kernel: Netfilter.....	30
`DROP` vs `REJECT`, and why it matters here.....	30
3.3 Architecture.....	30
3.4 Linux Internals.....	30
The full chain from kernel to ban.....	30
3.5 Implementation.....	31
Step 1 — Installation (both `backend01` and `backend02`).....	31
Step 2 — Jail configuration.....	31
Step 3 — Enable and start.....	32
Step 4 — Confirm the jail is active.....	32
3.6 Command Breakdown.....	32

3.7 Configuration Breakdown.....	32
3.8 Packet Flow.....	33
3.9 Process Flow.....	33
3.10 Verification.....	34
3.11 Failure Simulation.....	34
3.12 Troubleshooting.....	35
3.13 Production Engineering Notes.....	35
3.14 Interview Reference.....	36
Part 4 — rsync: Disaster Recovery.....	37
4.1 Engineering Problem.....	37
4.2 First Principles.....	37
Why not `scp`.....	37
The delta-transfer algorithm: rsync's actual innovation.....	37
Transport: rsync over SSH.....	37
Archive mode and metadata preservation.....	38
Why personal interactive keys break unattended backup.....	38
Why remote sudo elevation is required.....	38
Why Azure breaks `ssh-copy-id`.....	38
Exit code 24: the harmless warning rsync gets wrong.....	38
4.3 Architecture.....	39
4.4 Linux Internals.....	39
4.5 Implementation.....	39
Phase A — Manual Validation (run first, before any automation).....	39
Phase B — Hardened Automation (after Phase A validates connectivity).....	41
4.6 Command Breakdown.....	43
4.7 Configuration Breakdown.....	43
4.8 Packet Flow.....	44
4.9 Process Flow.....	44
4.10 Verification.....	44
4.11 Failure Simulation.....	45
4.12 Troubleshooting.....	46
4.13 Production Engineering Notes.....	46
4.14 Interview Reference.....	47
Part 5 — systemd Timers: Scheduled Automation.....	48
5.1 Engineering Problem.....	48
5.2 First Principles.....	48
What cron actually is and where it falls short here.....	48
What systemd timers provide.....	48
The five-field cron syntax (still worth knowing).....	49
The execution environment problem (a production failure worth documenting)....	49
5.3 Architecture.....	49

5.4 Linux Internals.....	49
5.5 Implementation.....	50
Step 1 — Create the service unit.....	50
Step 2 — Create the timer unit.....	50
Step 3 — Enable and start the timer.....	50
Step 4 — Confirm the timer is active and next activation is scheduled.....	50
5.6 Command Breakdown.....	51
5.7 Configuration Breakdown.....	51
5.8 Packet Flow.....	51
5.9 Process Flow.....	52
5.10 Verification.....	52
5.11 Failure Simulation.....	52
5.12 Troubleshooting.....	53
5.13 Production Engineering Notes.....	53
5.14 Interview Reference.....	54
Failure Lab Index — Cross-Technology Enterprise Troubleshooting Scenarios.....	55
Engineering Outcome.....	57
Final Engineering State.....	58
Operational Summary.....	59
Key Engineering Lessons.....	60

EdgeSecure Hybrid Infrastructure Lab

PHASE 5: Hybrid Overlay Networking, Centralized Logging, Intrusion Prevention, Disaster Recovery, and Scheduled Automation

1. Introduction

Phase 4 closed with a working Layer 7 load-balanced hybrid stack: `edge01` distributing traffic across `backend01` (local, VirtualBox) and `backend02` (cloud, Azure), with SELinux enforcing and HAProxy health checks tuned for the latency profile of the Azure link. The infrastructure works — but it works as three separate trust domains stitched together by a single load-balancing relationship. There is no private network connecting the nodes for anything other than HTTP. There is no centralized record of what happens on any of them. There is no automated defense against repeated attack. There is no disaster recovery posture. There is no scheduled automation layer.

Phase 5 closes all five of these gaps. It is the phase where EdgeSecure stops being "three servers that happen to load balance" and becomes infrastructure with an actual operational backbone: a private overlay network, a centralized audit trail, automated intrusion response, a recoverable backup strategy, and a scheduling layer that ties automation together.

Before any of that can be built, however, an incorrect assumption carried over from earlier planning has to be corrected on paper, because it would otherwise quietly break the entire centralized logging design.

2. Why the Original Plan Breaks: A First-Principles Look at Why Azure Cannot Reach 192.168.56.0/24

Before WireGuard is justified as a design decision rather than just a tool we like, it is worth proving — at the packet level — that the lab's original mental model was wrong, and exactly where it dies.

2.1 The assumption that needed correcting

Early planning assumed `backend02` (Azure) could simply forward its logs to `192.168.56.10`, the Host-only IP of `edge01`. This is the same address scheme used for SSH access from the local hypervisor host, so it felt natural to reuse it for log forwarding too. It is architecturally impossible, and the reason why is foundational to everything else in this phase.

2.2 What a Host-only adapter actually is

A VirtualBox Host-only network is not a real network in the sense of being reachable from anywhere outside the physical machine running VirtualBox. It is a virtual switch that VirtualBox creates entirely in software, connecting the hypervisor's host OS and its guest VMs. No physical NIC, no router, no ISP uplink, and critically, no Azure datacenter has any path into that switch. `192.168.56.10` only exists as a meaningful, routable address *inside the hypervisor's own memory and virtual switching fabric*. Azure has never heard of it and never will.

2.3 RFC 1918 and why this isn't just a VirtualBox quirk

192.168.0.0/16 (which contains **192.168.56.0/24**) is one of three blocks reserved by RFC 1918 as private address space, alongside **10.0.0.0/8** and **172.16.0.0/12**. Every internet backbone router in existence is configured — by design, by convention, and in many cases by hard policy — to silently discard any packet whose destination falls in these ranges if that packet arrives from outside a private network. This isn't a firewall rule that can be argued around; it's a structural property of how the global routing table (the Border Gateway Protocol mesh that makes "the internet" a single reachable graph) is built. Private addresses are deliberately excluded from that graph so that millions of independent LANs can all reuse the same address blocks without collision.

So even setting VirtualBox aside: even if **192.168.56.10** were somehow exposed on a real physical LAN, no router between Azure's datacenter in Spain Central and that LAN would forward a packet toward it. The packet would die at the first backbone hop that recognized the destination as RFC 1918 space arriving from outside a private context.

2.4 Tracing the packet's actual death

Walk the failure forward explicitly:

1. **backend02**'s rsyslog daemon constructs a packet with destination **192.168.56.10:514**.
2. The Linux kernel on **backend02** consults its local routing table. **192.168.56.0/24** is not a directly connected subnet and there is no static route for it, so the kernel falls back to the default route — **eth0**, out toward Azure's virtual network gateway.
3. Azure's network fabric receives the packet and attempts to forward it toward the public internet, because the destination is not part of Azure's own VNet address space either.
4. The first public-internet-facing router that handles the packet recognizes **192.168.56.0/24** as RFC 1918 private space and drops it. No ICMP error is even guaranteed to come back — it can simply vanish.
5. **backend02**'s rsyslog daemon, using UDP, never learns the packet failed. The log line is gone, with no error logged anywhere.

This last point matters operationally: a misconfiguration like this doesn't throw an obvious error. It fails *silently*. An engineer could believe centralized logging was working for weeks while **backend02**'s audit trail simply evaporated into the internet.

2.5 Why NAT alone doesn't fix this either

A reasonable next idea is "NAT it, then" — have **backend02** NAT its own outbound traffic so the source looks like a public IP. But NAT only rewrites *source* addressing as packets leave a boundary; it does nothing to make the *destination* **192.168.56.10** reachable, because that address still only exists inside a hypervisor's private virtual switch with no internet-facing presence at all. There is no public IP anywhere in the world that, if dialed, lands on **edge01**'s Host-only interface. The destination has to change, not just the source — which is precisely what an overlay network like WireGuard provides: it gives **edge01** a *second*, VPN-resident address that is genuinely reachable from **backend02**, because both ends explicitly agree to route to it through an encrypted tunnel rather than through the public backbone.

2.6 The corrected design

The fix is therefore not a firewall rule or a NAT table — it's a new Layer 3 overlay. **edge01** and **backend02** will each gain a **wg0** interface on a new private subnet, **10.100.0.0/24**, that exists purely as VPN-resident addressing. **backend01**, sitting on the same Host-only switch as

edge01, never needs WireGuard — it can keep forwarding logs straight to **192.168.56.10** as originally planned, because that path was always valid; it's only the Azure leg that was broken.

This is the constraint that shapes everything that follows: WireGuard is not an optional security enhancement bolted onto Phase 5 — it is the load-bearing fix that makes the rest of the phase's stated goal (centralized logging across the *entire* hybrid footprint) achievable at all.

3. Updated Lab Architecture

edge01 — Rocky Linux 9 (VirtualBox)

Roles: Bastion host, Ansible control node, HAProxy load balancer, WireGuard server, central rsyslog server.

Interfaces: **enp0s3** (NAT), **enp0s8** (**192.168.56.10**, Host-only), **wg0** (**10.100.0.1/24**, overlay — new in Phase 5).

backend01 — Ubuntu Server (VirtualBox, local)

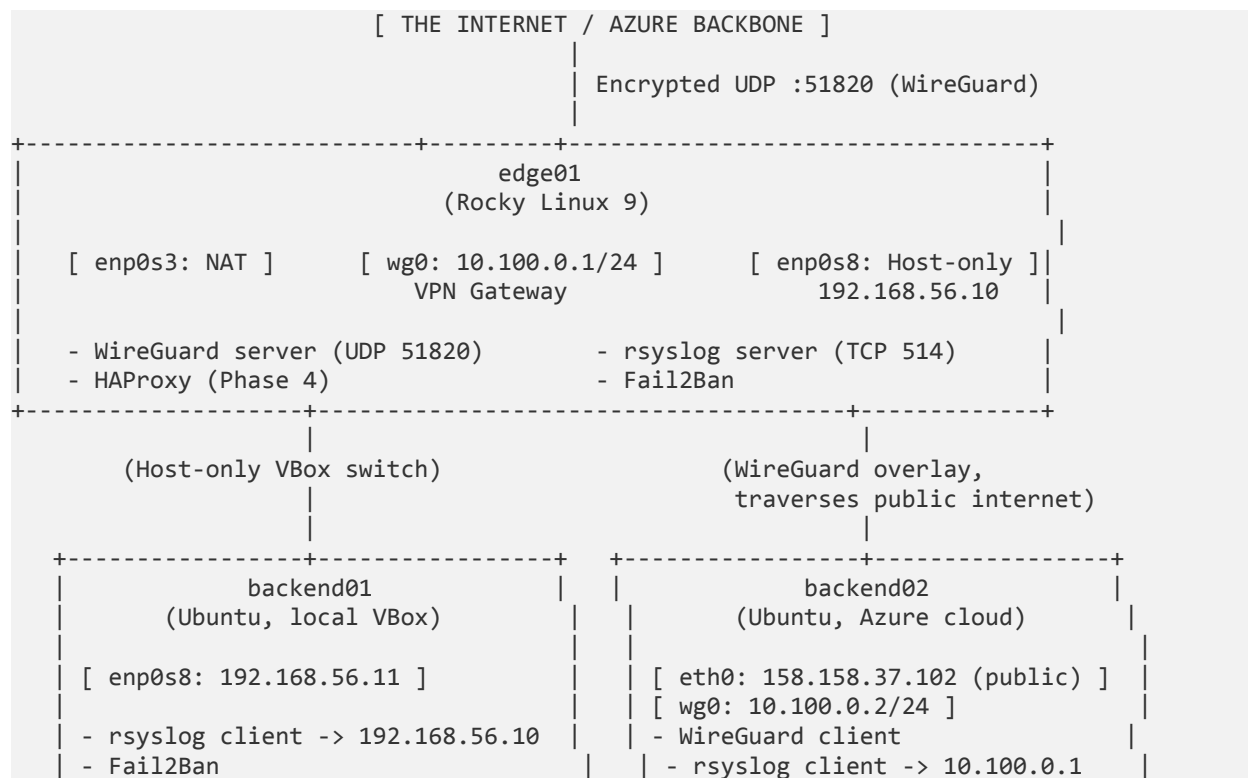
Roles: Managed application node, rsyslog client, Fail2Ban node.

Interfaces: **enp0s3** (NAT), **enp0s8** (**192.168.56.11**, Host-only). No WireGuard interface — it shares **edge01**'s Host-only switch directly, so no overlay is needed for it to reach the log server.

backend02 — Ubuntu Server (Azure, Spain Central)

Roles: Cloud application node, rsyslog client, Fail2Ban node, WireGuard client.

Interfaces: **eth0** (Azure VNet, public IP **158.158.37.102** — unchanged), **wg0** (**10.100.0.2/24**, overlay — new in Phase 5).



```
+-----+ | - Fail2Ban |
+-----+
```

Note the asymmetry in the diagram: **backend01** forwards logs over its existing Host-only path; **backend02** forwards logs *through the WireGuard tunnel*. Both ultimately land on **edge01**, but only one of them required a new Layer 3 overlay to get there — exactly per the routing analysis in Section 2.

Part 1 — WireGuard: Hybrid Overlay Networking

1.1 Engineering Problem

The infrastructure has two network domains that cannot natively reach each other: a VirtualBox Host-only switch (`192.168.56.0/24`), and an Azure Virtual Network exposing only a single public IP (`158.158.37.102`) for `backend02`. Section 2 proved that no amount of static routing, NAT, or firewall configuration makes `192.168.56.0/24` reachable from Azure — the address space is structurally excluded from the public internet's routing graph. We need a way to give `backend02` a *logical* presence on `edge01`'s private network without exposing that private network publicly, and without compromising the confidentiality or integrity of traffic that crosses the public internet to get there.

1.2 First Principles

A VPN's entire job is to solve exactly one structural problem: making private, non-routable address space reachable across a public, hostile transport, while guaranteeing that nothing on that public transport can read or alter the traffic. It does this through two distinct, separable functions that are easy to conflate:

- **Encapsulation** — wrapping a private packet (destination `10.100.0.x`) inside a public packet (destination `158.158.37.102`) so it can physically cross the internet's actual routing graph.
- **Encryption** — making the encapsulated payload unreadable and tamper-evident to anyone who intercepts it in transit.

These are conceptually independent. You could encapsulate without encrypting (a GRE tunnel does this, and is why GRE alone is not a security boundary). WireGuard couples both into a single kernel-resident mechanism.

It is worth being precise about why this is an *infrastructure* VPN problem rather than a consumer VPN problem, because the two get conflated. A consumer VPN (the kind sold to privacy-conscious individuals) exists to make *one client's* traffic appear to originate from a different egress point, hiding it from a local ISP or geographic restriction. An infrastructure VPN exists to make *two private networks* mutually reachable as if they were one contiguous LAN, with every node retaining its own identity rather than masquerading as something else. EdgeSecure needs the second kind: `backend02` needs an actual private address it can be addressed by, not an anonymized egress point.

Azure offers a managed product for this — Azure VPN Gateway — which would build a site-to-site IPsec tunnel between Azure's VNet and an on-premises gateway. It was deliberately not used here for three reasons relevant to this lab's goals: first, it requires a routable, stable public endpoint on the "on-premises" side with a static IP and a supported gateway device — `edge01` running inside a NATed VirtualBox host-only environment doesn't cleanly satisfy that. Second, it is a managed black box; the entire pedagogical point of this lab is to understand cryptographic routing and kernel packet handling from first principles, which a managed PaaS service deliberately abstracts away. Third, IPsec's own configuration surface (Phase 1/Phase 2 SAs, proposal matching, NAT-traversal quirks) is exactly the kind of brittle, footgun-prone surface WireGuard was built to eliminate, discussed further below.

1.3 Architecture

WireGuard introduces a new virtual network interface, `wg0`, on both `edge01` and `backend02`. This interface behaves like any other network interface from the kernel's perspective — it has an IP address, it appears in `ip addr`, and the routing table can send traffic to it — but unlike `eth0` or `enp0s3`, nothing physical backs it. Every packet that enters `wg0` is intercepted by the WireGuard kernel module before it goes anywhere, encrypted, and re-emitted out a *real* physical interface as a UDP datagram addressed to the peer's actual public endpoint.

`backend02` is configured as the listening side (`ListenPort = 51820`) because it has a stable, publicly reachable IP address (`158.158.37.102`) that `edge01` can dial into directly. `edge01` is configured as the initiating peer, because it sits behind VirtualBox NAT with no port forwarding configured — nothing on the public internet can reach `edge01` directly, but `edge01` can freely initiate outbound connections, which is all an initiating peer needs. `backend01` does not participate in the WireGuard mesh at all — it does not need to, because it already shares a Layer 2 broadcast domain with `edge01` via the Host-only switch.

This role assignment is not a preference or a convention — it is a hard constraint imposed by the network topology. An engineer setting up WireGuard must first answer "which side has a publicly reachable address?" and make that side the listener. Reversing the roles here would produce a configuration that looks syntactically valid but fails to establish a handshake, because `backend02` cannot dial into a VirtualBox NAT adapter with no port forwarding.

`PersistentKeepalive = 25` belongs on `edge01`'s side (the NAT-traversing initiator). NAT devices expire idle UDP session mappings after 30–120 seconds of inactivity. Without periodic keepalive traffic from `edge01`, `backend02` loses the ability to reach back in to `edge01` once that NAT mapping expires. `PersistentKeepalive = 25` on `edge01`'s peer block means it sends a small authenticated packet every 25 seconds, keeping its own NAT mapping alive indefinitely.

Critically — and this directly answers a requirement carried over from this phase's specification — **WireGuard does not replace `backend02`'s public IP.** `158.158.37.102` remains `eth0`'s address, used for SSH, HAProxy backend traffic, and any other existing flow. `wg0` is an *additional* interface with its *own* address (`10.100.0.2/24`), and the kernel routing table decides, per-packet, which interface a given destination should use. This is why WireGuard is additive rather than disruptive: nothing about Phase 4's existing reachability changes, because nothing about `eth0` changes.

1.4 Linux Internals

Why the kernel, not user space

Historically, the dominant Linux VPN technologies were OpenVPN (user-space) and IPsec (kernel-space but configured through a sprawling, multi-daemon stack: `strongSwan/racoon` for IKE negotiation, kernel XFRM for the actual packet transform). OpenVPN's architecture requires every packet to cross the user/kernel boundary: the kernel hands the packet to the OpenVPN daemon running as an ordinary process, that process performs the cryptographic transform in user space, then hands the result back to the kernel to actually transmit. Each crossing is a context switch — the CPU has to save the entire register state of what it was doing, switch into a different process's address space, and restore state again on the way back. At high packet rates this overhead compounds into measurable throughput loss and latency jitter.

WireGuard, authored by Jason A. Donenfeld and merged into the mainline Linux kernel as of version 5.6 (2020), eliminates that boundary crossing entirely. The cryptographic transform

happens inside the kernel module itself, in the same context that's already handling the packet's journey through the network stack. There is no daemon to schedule, no context switch, no copy of packet data across a user/kernel boundary purely for encryption purposes.

This kernel residency is also why WireGuard's codebase is dramatically smaller than alternatives — roughly 4,000 lines of code versus over 100,000 for a comparable OpenVPN + OpenSSL stack, and a comparably large surface for a full IPsec implementation. Donenfeld's explicit design philosophy was **cryptographic rigidity, not agility**: WireGuard supports exactly one suite of modern primitives, full stop, with no negotiation handshake to decide which cipher to use. IPsec's flexibility — supporting dozens of possible cipher and key-exchange combinations for backward compatibility — is precisely what has produced years of real-world misconfiguration vulnerabilities, because "what's the secure combination" becomes a question administrators have to get right rather than a question that doesn't need asking.

The cryptographic primitives, and why each was chosen

- **Curve25519** performs the Elliptic-Curve Diffie-Hellman key exchange that lets two peers, each holding a private key never transmitted anywhere, agree on a shared secret over a channel an attacker can fully observe. Curve25519 was selected over older NIST curves for resistance to a number of side-channel and implementation-pitfall classes that have plagued other elliptic curve choices.
- **ChaCha20** is the symmetric stream cipher that actually encrypts packet payloads once a shared secret exists. It was chosen over AES specifically because it performs well in pure software on platforms without dedicated AES hardware instructions (AES-NI), which matters for a kernel module meant to run uniformly across heterogeneous hardware, including lower-power and embedded targets.
- **Poly1305** is the Message Authentication Code paired with ChaCha20 (as the AEAD construction ChaCha20-Poly1305). It guarantees that any single bit flipped in transit — whether by network corruption or active tampering — is detected and the packet is discarded, rather than silently processed as if it were legitimate.
- **BLAKE2s** is used for hashing within the handshake.
- **The Noise Protocol Framework**, specifically its **IK** handshake pattern, defines the *sequencing* of messages that establishes a session: who sends what, in what order, and what each message proves. **IK** specifically means the initiator's static public key is transmitted as part of the very first handshake message (already known to the responder in advance), letting the handshake complete in a single round trip once both sides already know each other's public keys — which is exactly the EdgeSecure case, since both keys are configured ahead of time in `wg0.conf`.

Cryptokey Routing: the core abstraction

The single most important WireGuard-specific concept is **AllowedIPs**, and it does double duty in a way that's easy to underappreciate:

- **As an egress routing table**: when a packet enters `wg0` destined for some IP, WireGuard checks which configured peer's **AllowedIPs** list contains that destination, and encrypts the packet using *that peer's* key. This is the mechanism by which a single `wg0` interface can multiplex traffic to many different peers — the cryptographic identity *is* the routing decision.
- **As an ingress access-control filter**: when WireGuard decrypts an incoming packet, it checks the *inner* (decrypted) source IP against the sending peer's configured

AllowedIPs. If the inner source doesn't match, the packet is dropped — even though it decrypted successfully with a valid key. This stops a legitimate peer's compromised key from being used to spoof traffic claiming to originate from some other address.

This means authorization in WireGuard isn't a separate ACL bolted on after decryption — it's structurally fused with the decryption operation itself. There is no code path where a packet can be "decrypted but not yet authorized."

Why `PersistentKeepalive` matters here specifically

backend02 sits behind Azure's NAT/network fabric on its outbound path (even though it has a public IP, the path WireGuard's UDP traffic actually takes can still traverse stateful middleboxes). Stateful firewalls and NAT devices age out idle UDP flows, typically within 30–120 seconds of inactivity, because UDP has no connection teardown signal to tell them the flow ended — they have to guess based on a timeout. If that state entry expires, **edge01** cannot re-initiate contact with **backend02**, because the inbound path back through Azure's fabric requires that the connection state still exist. Setting **PersistentKeepalive = 25** on the client side means **backend02** sends a tiny, fully authenticated (but otherwise empty) packet every 25 seconds — comfortably under any realistic timeout — keeping the path's state alive in both directions indefinitely.

Kernel IP forwarding

By default, the Linux kernel's network stack behaves as a *host*: when it receives a packet not addressed to one of its own interface IPs, its instinct is to discard it, because hosts don't forward other people's traffic. **net.ipv4.ip_forward=1** flips this behavior at the kernel level, instructing the IPv4 stack to additionally consult its routing table for packets *not* destined locally, and forward them out the appropriate interface. This single sysctl is the difference between **edge01** being an endpoint and **edge01** being a router — and it has to be explicitly enabled, because routing capability is a meaningful security posture decision, not a default any host should silently have.

1.5 Implementation

Step 1 — Package installation (`edge01`)

```
sudo dnf install epel-release -y
sudo dnf install wireguard-tools -y
```

The **wireguard-tools** package supplies the userspace control utilities (**wg**, **wg-quick**) used to configure and inspect the kernel module. The actual **wireguard** kernel module itself ships mainline in kernel 5.6+, so on a current Rocky Linux 9 kernel no separate kernel module package is required — only the control-plane tooling.

Step 2 — Key generation (`edge01` and `backend02`, each independently)

```
umask 077
wg genkey | tee privatekey | wg pubkey > publickey
```

wg genkey produces a 256-bit Curve25519 private key, base64-encoded, written to standard output. The pipe into **tee privatekey** writes that same output to a file *and* passes it through to standard output simultaneously, which then feeds **wg pubkey**, deriving the corresponding public key and redirecting it into **publickey**. Setting **umask 077** first ensures the private key file is created with **600** permissions from the moment it exists, rather than briefly existing

world-readable before a subsequent `chmod` — a small but real race condition in security-sensitive file creation.

The private key never leaves the host it was generated on. Only the corresponding public key is shared with the peer.

Step 3 — Listener configuration (`backend02`)

`/etc/wireguard/wg0.conf` on `backend02`:

```
[Interface]
Address = 10.100.0.2/24
ListenPort = 51820
PrivateKey = <backend02_private_key>

[Peer]
# edge01
PublicKey = <edge01_public_key>
AllowedIPs = 10.100.0.1/32
```

`ListenPort = 51820` lives here because `backend02` is the side with a reachable public IP. `AllowedIPs = 10.100.0.1/32` uses `/32` — a single host — since `edge01` is the only peer authorized to appear as source `10.100.0.1` after decryption. No `Endpoint` line is needed on `backend02`; it learns `edge01`'s real source address dynamically from the first incoming handshake packet.

After saving the config, the Azure NSG must also permit inbound UDP 51820. The NSG is a separate firewall layer above the VM's OS — `ufw status` on `backend02` can show all-allowed while the NSG silently drops every packet before it ever reaches the NIC. In the Azure Portal: Network Interface → Network Security Group → Inbound security rules → add UDP port 51820, source Any.

Step 4 — Initiator configuration (`edge01`)

`/etc/wireguard/wg0.conf` on `edge01`:

```
[Interface]
Address = 10.100.0.1/24
PrivateKey = <edge01_private_key>

[Peer]
# backend02 (Azure)
PublicKey = <backend02_public_key>
Endpoint = 158.158.37.102:51820
AllowedIPs = 10.100.0.2/32
PersistentKeepalive = 25
```

No `ListenPort` on `edge01` — the initiating side does not need to bind a fixed port; the kernel assigns an ephemeral source port for outbound UDP automatically. `Endpoint` is required here because `edge01` must know where to send the initial handshake packet.

`PersistentKeepalive = 25` is on `edge01`'s side because `edge01` is the one behind NAT whose outbound mapping must be kept alive.

Critical: `PrivateKey` must contain the actual base64 key string — not a file path. Writing `PrivateKey = /home/eudofia/privatekey` instead of the key's contents causes `wg-quick` to fail immediately with "Key is not the correct length or format." Run `cat ~/privatekey` and paste the output.

Step 5 — IP forwarding and firewall zone (`edge01`)

```
echo "net.ipv4.ip_forward = 1" | sudo tee -a /etc/sysctl.d/99-wireguard.conf
sudo sysctl -p /etc/sysctl.d/99-wireguard.conf
```

Since `edge01` is the initiating peer it does not need `firewalld` to open UDP 51820 inbound — it only sends outbound, and connection tracking handles the return path. What it does need is for `wg0` to be placed in a zone that permits the overlay traffic intended to flow through the tunnel. Otherwise, decrypted traffic arriving from the tunnel hits `edge01`'s existing restrictive zone rules and gets dropped — the tunnel can be healthy (handshake complete, ping working) while TCP applications like `rsyslog` simultaneously fail:

```
sudo firewall-cmd --zone=internal --change-interface=wg0 --permanent
sudo firewall-cmd --zone=internal --add-protocol=icmp --permanent
sudo firewall-cmd --reload
```

Verify the zone assignment took effect before proceeding:

```
sudo firewall-cmd --get-zone-of-interface=wg0
sudo firewall-cmd --zone=internal --list-all
```

Port 514/tcp for `rsyslog` will be added to this zone explicitly in Part 2. Do not use the `trusted` zone here — it permits all traffic with no restriction. The `internal` zone allows deliberate, additive port grants while maintaining a security posture appropriate for traffic that, while authenticated at the WireGuard layer, still deserves explicit service-level access control.

Step 6 — Bring the interface up (both nodes)

```
sudo systemctl enable --now wg-quick@wg0
```

`wg-quick` is a shell script wrapper that reads `wg0.conf`, creates the `wg0` interface via `ip link`, assigns its address via `ip addr`, loads the peer configuration via `wg setconf`, and brings the interface up via `ip link set up` — all the discrete steps a manual deployment would otherwise require individually.

1.6 Command Breakdown

<code>wg genkey</code>	Generates a Curve25519 private key	Userspace only; pure key material generation, no kernel interaction
<code>wg pubkey</code>	Derives public key from private key via scalar multiplication on the curve	Userspace only
<code>wg-quick up wg0</code>	Brings the full interface to life from a config file	Kernel netlink interface creation, IP assignment, peer table population
<code>wg show</code>	Queries live interface and peer state	Kernel netlink socket, read-only

<code>wg set</code>	Modifies peer configuration live, without restarting the interface	Kernel netlink socket, write
<code>sysctl net.ipv4.ip_forward</code>	Reads/writes the kernel's forwarding behavior flag	<code>/proc/sys/net/ipv4/ip_forward</code> , a direct window into kernel state

1.7 Configuration Breakdown

- **Address = 10.100.0.1/24** — the IP assigned to the local `wg0` interface, in CIDR notation. The `/24` here defines this node's *local* view of the subnet mask, used for the kernel's own routing table entry — it does not, by itself, grant reachability to anything; reachability is governed entirely by **AllowedIPs** in the peer sections.
- **ListenPort = 51820** — the UDP port the kernel binds to for inbound WireGuard traffic. This is configurable but 51820 is the de facto WireGuard convention.
- **PrivateKey** — this node's own private key. Never the peer's.
- **[Peer] PublicKey** — the *remote* peer's public key, used to verify and decrypt anything claiming to be from that peer.
- **Endpoint** — only meaningful on the *initiating* side (`edge01`). It tells WireGuard the actual outer-layer socket (IP:port) to send UDP datagrams to — in this lab, `158.158.37.102:51820`, `backend02`'s real public address. The listening side (`backend02`) does not need this in its peer block; it learns `edge01`'s real source endpoint dynamically from the first incoming packet.
- **AllowedIPs** — covered exhaustively in 1.4; the single most consequential line in the file.
- **PersistentKeepalive = 25** — set on `edge01`'s peer block, not `backend02`'s, because `edge01` is the side behind NAT. `backend02`, as the listener with a stable public IP, does not need to keep any NAT mapping alive — it is always directly reachable. `edge01` must send periodic keepalives to prevent its own NAT device from expiring the outbound UDP session mapping.

1.8 Packet Flow

Tracing a single ICMP echo request from `backend02` to `edge01`'s overlay address:

1. **Application layer:** `ping 10.100.0.1` constructs an ICMP Echo Request with destination `10.100.0.1`.
2. **Routing table lookup:** the kernel on `backend02` consults its routing table. The `wg-quick up` step installed a route stating `10.100.0.0/24 via wg0`. The packet is handed to the `wg0` interface.
3. **WireGuard interception:** the kernel's WireGuard implementation receives the packet on `wg0` before it would ever reach a physical NIC. It checks its configured peers' **AllowedIPs** and finds that `10.100.0.1` matches the `edge01` peer entry.
4. **Encryption:** the original IP packet (header and payload together) is encrypted as a single opaque blob using the session key derived from the Noise handshake, and authenticated with Poly1305.

5. **Encapsulation:** this encrypted blob becomes the *payload* of a brand-new UDP packet. The new packet's IP header has source `backend02`'s real address and destination `edge01`'s actual endpoint address (whatever was configured in `Endpoint`), destination port `51820`.
6. **Physical transmission:** this new UDP packet is handed to the normal routing path and exits via `eth0`, traversing Azure's network fabric and the public internet exactly like any other UDP packet — because to every router along the path, it *is* just an ordinary UDP packet. The encrypted payload inside is opaque to all of them.
7. **Arrival at `edge01`:** the UDP packet arrives on `enp0s3`, addressed to UDP port 51820, where the kernel's WireGuard module is bound and listening.
8. **Decryption and verification:** WireGuard decrypts the payload, recovering the original ICMP packet, and checks the decrypted *inner* source address (`10.100.0.2`) against the sending peer's configured `AllowedIPs`. It matches, so the packet is accepted.
9. **Re-injection:** the now-decrypted, original ICMP packet is injected into the kernel's normal packet-processing path as if it had simply arrived on `wg0` directly — because, from the kernel's perspective downstream of WireGuard, it did.
10. **Delivery:** the ICMP stack on `edge01` sees the Echo Request addressed to its own `wg0` IP (`10.100.0.1`) and responds with an Echo Reply, which retraces the same encryption/encapsulation process in reverse.

1.9 Process Flow

On the control-plane side, distinct from any single packet's journey:

1. `wg-quick up wg0` is invoked (directly or via `systemd` at boot).
2. The script parses `wg0.conf`.
3. A new virtual network device is created via a netlink `RTM_NEWLINK` request, of kind `wireguard` — this is a kernel-resident device type, not a TUN/TAP device requiring a userspace process to service it.
4. The interface's address is assigned via `RTM_NEWADDR`.
5. Peer configuration (public keys, allowed IPs, endpoints, keepalive intervals) is pushed into the kernel module's internal peer table via the WireGuard-specific netlink interface (the same interface `wg set` and `wg show` use).
6. The interface is brought administratively up via `RTM_NEWLINK` with `IFF_UP`.
7. Static routes for each peer's `AllowedIPs` are installed into the routing table, pointing at `wg0`.
8. From this point forward, the kernel module operates autonomously — no userspace daemon needs to remain running for traffic to flow; `wg-quick` and `wg` are purely configuration-time tools.

1.10 Verification

```
sudo wg show
```

`wg` communicates with the kernel via a generic netlink socket dedicated to the WireGuard module, requesting current interface and peer state. The critical field to inspect is `latest handshake`. A populated, recent timestamp means the Noise `IK` handshake has completed and a session key is active. A blank or stale value with `transfer: 0 B received` means no valid

traffic has been exchanged — keys may be misconfigured, the port may be blocked, or the peer may not be attempting to connect at all.

```
ping -c 4 10.100.0.1
```

From **backend02**, this exercises the full packet flow described in 1.8 in both directions. A successful reply confirms: correct key configuration on both sides, correct **AllowedIPs** matching, UDP 51820 reachability through any intermediate firewalls (including Azure NSGs, which must also permit egress/ingress on that port), and correct kernel forwarding/interface state on both ends.

```
ip route show table main | grep wg0
```

Confirms the kernel actually installed the expected route for the overlay subnet pointing at **wg0** — a missing entry here, even with a healthy handshake, would mean traffic destined for the overlay subnet is taking some other path (or none).

1.11 Failure Simulation

Break 1 — Misconfigured `AllowedIPs` on the server side.

Change **edge01**'s peer entry to **AllowedIPs = 10.100.0.99/32** (a deliberately wrong host).

Symptom: **wg show** on **edge01** shows zero handshake activity, or a handshake that completes once but then no actual application traffic passes.

Root cause: even if the Noise handshake itself can complete (handshake messages are tied to the peer's public key, not yet to **AllowedIPs**), any *data* packet whose decrypted inner source is **10.100.0.2** gets silently dropped because it doesn't match the configured **10.100.0.99/32**.

Diagnosis: **wg show** reveals a handshake timestamp but **transfer** counters that never grow past the handshake's own overhead.

Fix: correct the **AllowedIPs** entry and either restart the interface or use **wg set wg0 peer <pubkey> allowed-ips 10.100.0.2/32** to patch it live without dropping the tunnel.

Break 2 — UDP 51820 blocked upstream of `edge01` (e.g., a hypervisor-level or cloud NSG rule, not just `firewalld`).

Symptom: **wg show** on **backend02** never shows a handshake at all, ever — not a stale one, an *absent* one.

Root cause: the very first handshake-initiation UDP packet from **backend02** never reaches **edge01**'s kernel, so the WireGuard module on **edge01** has nothing to respond to. There is a meaningful difference between "handshake completed once, then traffic stopped" (a routing/AllowedIPs problem, downstream of the kernel module) and "no handshake ever recorded" (a reachability problem, upstream of the kernel module entirely).

Diagnosis: **tcpdump -i enp0s3 udp port 51820** on **edge01** while **backend02** attempts a connection. If nothing arrives there at all, the problem is firmly between the two hosts at the network layer, not in WireGuard's configuration.

Fix: work outward from **edge01**: confirm **firewall-cmd --list-ports** shows **51820/udp**, then confirm any cloud-level network security group governing **edge01**'s VirtualBox host's actual internet path, if applicable, and finally confirm **backend02**'s own outbound NSG rules in Azure permit egress on that port.

Break 3 — `PersistentKeepalive` omitted on the client side, with an intervening NAT.

Symptom: the tunnel works perfectly when `backend02` initiates traffic, but `edge01` can never reach `backend02` first — for example, if a future phase needs `edge01` to push something to `backend02` unprompted, those connections silently fail until `backend02` sends *something* again.

Root cause: the NAT/stateful device's mapping for the UDP flow expired after a period of `backend02`-side inactivity, removing `edge01`'s ability to reach back in.

Diagnosis: handshake timestamps that go stale specifically during periods where `backend02` itself initiated nothing, then recover the moment `backend02` sends any traffic again.

Fix: add `PersistentKeepalive = 25` to `backend02`'s peer block.

1.12 Troubleshooting

A disciplined WireGuard troubleshooting order, from outermost to innermost:

1. Is the interface even up? (`ip link show wg0`)
2. Is the correct address assigned? (`ip addr show wg0`)
3. Is the UDP port reachable at all, end to end, ignoring WireGuard entirely? (`nc -u -z -v <endpoint> 51820` from the client, or packet capture on the server)
4. Has a handshake ever completed? (`wg show` — handshake timestamp present or absent)
5. Is a handshake completing repeatedly but no data flowing? (`AllowedIPs` mismatch, most likely)
6. Is data flowing on `wg0` but not reaching the application? (now it's no longer a WireGuard problem — it's ordinary kernel routing, `ip_forward`, or the destination application's own bind address)

This ordering matters because each layer's symptoms can superficially resemble the layer below it, and working from the wrong end wastes significant diagnostic time — a `sysctl` problem and an `AllowedIPs` problem can both present as "ping doesn't work," but the evidence that distinguishes them (handshake state) is only visible if you check it explicitly rather than assuming.

1.13 Production Engineering Notes

In a real enterprise deployment, several refinements beyond this lab's scope would typically follow:

- **Key rotation policy:** WireGuard has no built-in key expiry or rotation mechanism — keys are valid until manually replaced. Production environments typically script periodic rotation via configuration management (Ansible, in this lab's existing toolchain) rather than relying on indefinite key lifetime.
- **Mesh vs. hub-and-spoke:** this lab uses a simple two-node, hub-and-spoke overlay. Larger fleets often need either a full mesh (every node peers with every other node directly) or a routed hub design (all traffic flows through a central node, as here) — the choice trades off blast radius of a single compromised node against operational complexity of key distribution.
- **MTU considerations:** WireGuard's encapsulation overhead (roughly 60 bytes for the UDP/IP/WireGuard headers combined) reduces the effective MTU available to

encapsulated traffic. Production deployments crossing networks with unusual MTU constraints (some cloud provider virtual networks impose lower-than-1500-byte effective MTUs) need to explicitly tune `MTU =` in `wg0.conf` to avoid fragmentation-related performance cliffs.

- **Monitoring:** `wg show` output should be scraped into a metrics pipeline (Phase 6 SIEM territory) so that a silently-dead tunnel — one where the interface is technically "up" but the handshake has gone stale — triggers an alert rather than being discovered the next time someone manually checks.

1.14 Interview Reference

Q: Why does WireGuard outperform OpenVPN at scale?

A: WireGuard's cryptographic operations execute inside the kernel, in the same context already processing the packet, eliminating the user/kernel context switches that OpenVPN's userspace daemon architecture requires for every packet. At high packet rates, this difference compounds into materially better throughput and lower, more consistent latency.

Q: What is Cryptokey Routing and why does it matter for security, not just routing?

A: It's the fusion of `AllowedIPs` as both the routing decision (which peer's key to encrypt with) and the access-control decision (which decrypted source addresses are accepted from a given peer). Because authorization is structurally inseparable from decryption, there's no code path where a packet can be valid-but-unauthorized — a category of bug that has caused real vulnerabilities in systems where routing and access control are implemented as separate, occasionally-inconsistent layers.

Q: A WireGuard peer shows a stale handshake. What's your first diagnostic step, and why that one specifically?

A: Confirm raw UDP reachability on the configured port before touching any WireGuard-specific configuration — a stale handshake with zero received bytes points to a network-layer reachability problem (firewall, NSG, NAT) that no amount of `wg0.conf` editing will fix, whereas jumping straight to key/AllowedIPs changes wastes time if the real problem is that the packets never arrive at all.

EOF

echo "WireGuard part complete"

Part 2 — rsyslog: Centralized Logging

2.1 Engineering Problem

Each node currently keeps its own logs locally, in isolation. This creates two distinct operational failures. First, an investigative one: diagnosing any issue that spans multiple nodes (a failed health check on `backend02` that correlates with an auth failure on `backend01`, for instance) requires an engineer to authenticate into every node individually and manually correlate timestamps across disparate files — a process that doesn't scale past a handful of hosts and is exactly the kind of "memorized procedure" this lab's philosophy explicitly rejects in favor of designed-in visibility. Second, a forensic one: local logs are not tamper-resistant. If any node is compromised, an attacker's very first move is routinely to erase or alter `/var/log/auth.log` or `/var/log/secure` to remove evidence of their own access — and if that's the only copy, the evidence is simply gone.

Centralized logging solves both: a single durable destination for every node's event stream, written immediately and continuously, so that even total compromise or destruction of a node leaves its history intact elsewhere.

2.2 First Principles

journald vs rsyslog — two different jobs, not competing tools

`systemd-journald` is the modern default logging subsystem on most current Linux distributions, including both Rocky Linux 9 and current Ubuntu Server releases. It captures structured log data — from the kernel ring buffer, from any `systemd`-managed service's stdout/stderr, and from anything writing to its socket — and stores it in an indexed, binary format on disk, queryable locally via `journalctl` with rich filtering (by unit, by time range, by priority) that plain text `grep` can't match in performance at scale. What `journald` does *not* natively do is reliably forward that data to a remote network destination as its primary function — that's a deliberately separate concern.

`rsyslog` — "the rocket-fast system for log processing" — is the layer that handles that separate concern. It can ingest from `journald`, from the traditional `/dev/log` Unix domain socket, or directly from the kernel, apply filtering and routing rules, and crucially, transmit the resulting log stream over a real network transport (TCP or UDP) to a remote collector. The two systems are complementary, not redundant: `journald` is the structured local store and query engine; `rsyslog` is the routing and network-forwarding layer that gives that data an existence beyond the host that generated it.

`/dev/log` and the Unix domain socket model

Historically — and still today, for compatibility — most Linux logging starts at a Unix domain socket at `/dev/log`. Any process calling the C library's `syslog()` function is, under the hood, opening a connection to this socket and writing a formatted message into it. This is a purely local IPC mechanism: no network stack involvement, no port, just the kernel moving bytes between two processes on the same machine via a special filesystem-resident socket type. On a modern system, `journald` is typically the process actually listening on `/dev/log`, intercepting these legacy syslog-API writes and folding them into its structured journal alongside everything else it collects. `rsyslog`, in turn, can be configured to read back out of the journal (via

`imjournal`) or to listen on `/dev/log` directly itself, depending on configuration — in this lab, we let `journald` own `/dev/log` as is standard, and have `rsyslog` consume from the journal.

Why TCP, deliberately, over UDP

UDP-based syslog (the historical default, port 514/udp) is "fire and forget": the sending host constructs a UDP datagram and transmits it with no acknowledgment, no retry, and no awareness of whether it arrived. If a packet is dropped anywhere along the path — a momentary network blip, a router under load, in this lab's case very plausibly Azure-to-on-premises latency or transient loss — that log line is gone permanently, with the sender never even aware it happened. For an audit trail, an architecture that can silently and invisibly lose records is close to worthless; you cannot trust a security log you cannot trust is complete.

TCP-based syslog (514/tcp, the convention this lab uses) establishes an actual connection with sequencing and acknowledgment. If a packet is lost, TCP's own retransmission logic recovers it transparently, below the application layer. If the connection itself drops entirely (the receiving end goes offline, or a network partition occurs), `rsyslog` detects this at the application layer and can be configured to *queue* outgoing messages to local disk rather than discarding them, replaying the backlog once connectivity returns — a capability fundamentally unavailable to a UDP-based design, since UDP has no notion of "the connection is down" to even react to.

Given that this lab's hybrid architecture explicitly includes a WAN hop with the latency and occasional instability characteristics already documented in Phase 4's HAProxy work, choosing UDP for the audit trail would be choosing to silently lose security-relevant records during exactly the kind of transient network event the lab has already proven happens. TCP is the only choice consistent with the stated forensic goal.

Why port 514

514 is the IANA-registered port for syslog, used by both the legacy UDP convention and its TCP analogue. There's no deep technical requirement to use 514 specifically — `rsyslog` can bind to any port — but using the registered convention means any future tooling, firewall documentation, or engineer onboarding to this infrastructure encounters the expected default rather than a bespoke, undocumented choice that has to be separately remembered.

Why `/var/log/remote/` must exist as a structured hierarchy, not just a flat dump

A naive implementation could simply append every incoming line to one shared file. This fails in three concrete ways. First, attribution: without separating logs by source host, an engineer reviewing the file cannot quickly determine which node generated a given line without parsing the message body itself for hints, which not every log format reliably provides. Second, retention and rotation: log rotation tools (`logrotate`) operate per-file; a single giant shared file either rotates everything at once (losing the ability to retain different hosts' histories on different schedules) or can't be sensibly rotated at all without an external split step. Third, access control: in a multi-tenant or multi-team environment, restricting which engineers can read which host's logs is straightforward when logs are filed per-host-per-directory, and effectively impossible when they're interleaved in one file.

`rsyslog`'s dynamic file templates solve this by constructing the destination path *from the metadata of the incoming message itself* — specifically the sending host's name and the originating program — so that the filing happens automatically and correctly for every message, including from hosts that don't exist yet at configuration time. This is why the directive is a

template, not a static path: the directory structure has to scale to however many hosts eventually report in, without ever touching the configuration file again.

2.3 Architecture

`edge01` runs the rsyslog server, listening on TCP 514, writing incoming logs into `/var/log/remote/<hostname>/<program>.log`. Both `backend01` and `backend02` run rsyslog purely as forwarding clients — they retain their own local journal as already managed by `journal`, and *additionally* forward a copy of everything to `edge01`.

The forwarding destination differs by node, directly following from Section 2's routing analysis:

- `backend01` forwards to `192.168.56.10:514` — the Host-only address — because it shares that Layer 2 segment with `edge01` directly. No overlay involvement needed; this path was never broken.
- `backend02` forwards to `10.100.0.1:514` — `edge01`'s WireGuard overlay address — because, as proven in Section 2, the Host-only address is fundamentally unreachable from Azure, and the overlay is the only valid path that exists.

2.4 Linux Internals

The local message path on each client, before anything reaches the network, runs through several discrete layers worth naming explicitly. An application calling `syslog()` writes into `/dev/log`. On a system where `journal` owns that socket, the message is parsed, timestamped, tagged with structured metadata (PID, UID, cgroup, boot ID, and more), and appended to the binary journal on disk. `rsyslog`, configured with the `imjournal` input module, polls or subscribes to journal changes and pulls new entries into its own internal processing pipeline. From there, rsyslog's rule engine — historically a line-oriented filter language, now more commonly RainerScript in modern configurations — evaluates each message against configured selectors (`*.*` meaning "every facility, every priority") and, for matching messages, executes the configured *action*: in this case, forwarding over a TCP output module (`omfwd`).

On the server side, the inbound TCP module `imtcp` is loaded explicitly via `module(load="imtcp")` — unlike `imjournal`, which is typically loaded by distribution default, `imtcp` is not, because listening on a network port is a deliberate security posture decision a distribution shouldn't make on a user's behalf silently.

Reliable delivery and disk-assisted queues

A subtlety worth making explicit: TCP transport guarantees reliable delivery of bytes that are *actually sent*, but does nothing for messages generated while the TCP connection itself is down. For that gap, rsyslog maintains its own application-layer queuing — by default, an in-memory queue, but configurable as a Disk-Assisted Memory Queue, which spills to disk once the in-memory queue fills or the connection has been down longer than expected, and replays the spooled backlog once connectivity is restored. This is a second, independent reliability layer sitting *above* TCP's own retransmission guarantees — TCP guarantees the bytes you handed it arrive; rsyslog's queueing guarantees you have bytes to hand it even during an outage.

2.5 Implementation

Step 1 — Server configuration (`edge01`)

/etc/rsyslog.d/10-remote-input.conf:

```
module(load="imtcp")
input(type="imtcp" port="514")

template(name="RemoteLogs" type="string"
  string="/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log")

if $fromhost-ip != '127.0.0.1' then {
  action(type="omfile" dynaFile="RemoteLogs")
  stop
}

sudo mkdir -p /var/log/remote
sudo chown syslog:adm /var/log/remote # adjust ownership to the rsyslog service account
sudo systemctl restart rsyslog

sudo firewall-cmd --add-port=514/tcp --permanent
sudo firewall-cmd --reload
```

Step 2 — Client configuration (*backend01*)

/etc/rsyslog.d/60-remote-forward.conf:

```
.* action(type="omfwd" target="192.168.56.10" port="514" protocol="tcp"
  queue.type="LinkedList" queue.filename="fwd_q01"
  action.resumeRetryCount="-1" queue.saveOnShutdown="on")
```

Step 3 — Client configuration (*backend02*)

/etc/rsyslog.d/60-remote-forward.conf:

```
.* action(type="omfwd" target="10.100.0.1" port="514" protocol="tcp"
  queue.type="LinkedList" queue.filename="fwd_q02"
  action.resumeRetryCount="-1" queue.saveOnShutdown="on")

sudo systemctl restart rsyslog # on both backend01 and backend02
```

2.6 Command Breakdown

<code>module(load="imtcp")</code>	Loads the inbound TCP listener module into the running rsyslog instance
<code>input(type="imtcp" port="514")</code>	Binds that module to a specific port
<code>template(...)</code>	Defines a reusable, dynamically-evaluated output path
<code>action(type="omfile" dynaFile=...)</code>	Writes matching messages to a file path computed from the template at runtime, per message
<code>action(type="omfwd" ...)</code>	The forwarding action; <code>omfwd</code> is rsyslog's output module specifically for sending to a remote syslog destination

<code>queue.type="LinkedList"</code>	Configures an in-memory queue backing the forwarding action, allowing buffering rather than synchronous blocking per message
<code>action.resumeRetryCount="-1"</code>	Instructs rsyslog to retry an action indefinitely on failure, rather than giving up after a fixed number of attempts — appropriate for a destination expected to occasionally be briefly unreachable rather than permanently gone

2.7 Configuration Breakdown

- `%HOSTNAME%` / `%PROGRAMNAME%` — rsyslog template properties, populated per-message from the metadata the sending host attached to that log line, not from any static configuration. This is precisely what lets the directory structure scale to hosts that don't exist at config-write time.
- **stop** — halts further rule processing for a message once it's been filed to its remote-derived path, preventing it from *also* falling through to `edge01`'s own default local logging rules and cluttering `edge01`'s native `/var/log/messages` with every other host's traffic.
- `queue.saveOnShutdown="on"` — ensures that any messages still sitting in the in-memory queue at the moment rsyslog is stopped or restarted are flushed to disk first rather than discarded, so a routine service restart on the client doesn't itself become a silent loss event.

2.8 Packet Flow

Tracing a single log line generated on `backend02` by an administrator running `logger "test"`:

1. `logger` writes the message to `/dev/log`.
2. `journald`, which owns that socket, timestamps and stores the entry in the local binary journal.
3. rsyslog's `imjournal` input picks up the new journal entry.
4. rsyslog's rule engine evaluates it against the `*.*` selector in `60-remote-forward.conf` — matches everything, so it does.
5. The `omfwd` action constructs a TCP payload (the formatted syslog message) and either reuses an existing TCP connection to `10.100.0.1:514` or opens a new one if none exists.
6. The kernel routing table on `backend02` sees `10.100.0.1` falls within the `10.100.0.0/24` route installed by WireGuard, and hands the packet to `wg0`.
7. WireGuard encrypts and encapsulates it (exactly as detailed in Part 1, Section 1.8), and it crosses the public internet as an ordinary UDP packet on port 51820.
8. `edge01` receives, decrypts, and re-injects the original TCP segment, which is now addressed to `10.100.0.1:514` — a socket `edge01`'s rsyslog server is actually bound to and listening on.

9. rsyslog's `imtcp` module accepts the data, parses it as a syslog message, and evaluates it against the `RemoteLogs` template rule, computing the destination path `/var/log/remote/backend02/logger.log` (or the relevant program name) from the message's own embedded hostname field.
10. The line is appended to that file on disk.

2.9 Process Flow

At the daemon level, independent of any single message: `rsyslog.service` starts at boot via `systemd` on all three nodes. On `edge01`, it loads `imtcp` and binds to port 514 — a privileged port, meaning rsyslog must either start with sufficient capability to bind it (historically running briefly as root before dropping privileges, or holding `CAP_NET_BIND_SERVICE` explicitly) before continuing to run as its unprivileged service account for actual message processing. On the clients, rsyslog establishes its outbound TCP connection (or queues to attempt one) as soon as it starts, and maintains it persistently rather than reconnecting per message — repeatedly tearing down and rebuilding a TCP connection for every single log line would impose a wholly unnecessary handshake cost per message.

2.10 Verification

```
# On edge01:
ls -la /var/log/remote/

# Generate test traffic on backend02:
logger "Phase 5 connectivity validation from backend02"

# Back on edge01:
cat /var/log/remote/backend02/logger.log

# Confirm the TCP listener is actually bound:
sudo ss -tlnp | grep 514
```

`ss` (socket statistics) queries the kernel's socket tables directly, confirming rsyslog genuinely holds an open listening socket on the expected port — distinct from merely confirming the *configuration file* says it should.

2.11 Failure Simulation

Break 1 — Firewall blocks 514/tcp on `edge01` after a config change.

Symptom: clients show queued, unsent messages accumulating; `edge01` shows nothing new arriving.

Diagnosis: `ss -tlnp | grep 514` on `edge01` still shows the listener bound locally — the problem is reachability, not the daemon. `tcpdump -i wg0 tcp port 514` on `edge01` during a test from `backend02` shows nothing arriving, confirming the block is upstream of the application.

Fix: `firewall-cmd --add-port=514/tcp --permanent && firewall-cmd --reload`.

Break 2 — SELinux denies rsyslog from writing into `/var/log/remote/`.

Symptom: rsyslog service is healthy, the TCP connection is healthy, but `/var/log/remote/` stays empty or only partially populated.

Diagnosis: `sudo ausearch -m avc -ts recent` reveals an AVC denial for the rsyslog process attempting a write operation on a path it doesn't have an appropriate SELinux file context for, since `/var/log/remote/` is a non-standard path SELinux's default policy has no specific rule for.

Fix: `sudo semanage fcontext -a -t var_log_t "/var/log/remote(/.*)?"` followed by `sudo restorecon -Rv /var/log/remote`, applying the standard log-directory SELinux context to the custom path so the policy permits the write.

Break 3 — `omfwd` queue file permission errors.

Symptom: client-side rsyslog logs (visible via `journalctl -u rsyslog`) show repeated errors about being unable to write the disk queue file.

Root cause: the queue filename directory either doesn't exist or isn't writable by the rsyslog service account, meaning any connectivity gap *cannot* be buffered to disk at all — defeating the entire purpose of configuring a disk-assisted queue in the first place.

Fix: confirm the rsyslog working directory (commonly `/var/spool/rsyslog`) exists with correct ownership before relying on disk-assisted queueing in production.

2.12 Troubleshooting

A layered approach mirroring Part 1's: first confirm the daemon is running and bound (`systemctl status rsyslog, ss -tlnp`); then confirm raw network reachability independent of rsyslog entirely (`nc -vz <target> 514`); then confirm the application-layer connection is actually being attempted and accepted (`journalctl -u rsyslog` on both ends, and `tcpdump` on the relevant interface); then, only once transport is confirmed healthy, look at filing/templating logic (SELinux contexts, directory permissions, template syntax) as the remaining candidate.

2.13 Production Engineering Notes

- **Log rotation:** `/var/log/remote/` needs its own `logrotate` configuration distinct from the host's own default, since the directory structure and expected growth rate differ entirely from local system logs.
- **TLS:** this lab's TCP transport is reliable but not confidential beyond what WireGuard's tunnel already provides for the `backend02` leg — `backend01`'s traffic over the Host-only switch is unencrypted at the rsyslog layer, relying entirely on that switch's physical isolation. A hardened production deployment would typically layer `omfwd`'s native TLS support (via `gtls` or `oss1` stream drivers) on top, so log confidentiality doesn't depend entirely on network topology assumptions remaining true forever.
- **Retention and storage growth:** centralizing logs concentrates storage growth onto a single node (`edge01`). Production deployments monitor `/var/log/remote/`'s disk usage explicitly and typically ship aged data to cheaper, larger cold storage rather than letting it grow indefinitely on the same volume serving as the bastion's primary disk.

2.14 Interview Reference

Q: Why TCP instead of UDP for centralized syslog in this architecture specifically?

A: The hybrid link between Azure and the local environment has already demonstrated, in Phase 4's HAProxy work, latency and reliability characteristics that make transient loss

plausible. UDP syslog has no mechanism to detect or recover from a dropped packet, meaning exactly the kind of network event this lab's own infrastructure is prone to would silently and permanently destroy audit records. TCP, paired with rsyslog's own disk-assisted queueing above it, survives both transient packet loss and full connection outages without data loss.

Q: Why does `backend02`'s log traffic need WireGuard but `backend01`'s doesn't?

A: `backend01` shares a Layer 2 Host-only network segment directly with `edge01`, so `192.168.56.10` has always been a valid, reachable destination for it. `backend02` sits in Azure, where that same address is RFC 1918 private space with no path through the public internet's routing graph at all — WireGuard's overlay is the only mechanism that gives `backend02` a genuinely reachable address for `edge01`.

Part 3 — Fail2Ban: Automated Intrusion Response

3.1 Engineering Problem

Both `backend01` and `backend02` expose SSH, and `backend02` additionally exposes a public IP directly to the internet. The moment any service is internet-reachable, it becomes a target for automated credential-stuffing and brute-force scanning — not hypothetically, but as a constant, measurable background condition of running anything on the public internet. SSH hardening from earlier phases (key-based auth, no password login) already removes the *successful* attack surface, but it does nothing to stop the *attempt traffic* itself from consuming connection slots, filling logs with noise that obscures genuine signal, and consuming CPU on repeated handshake/rejection cycles. We need a mechanism that observes failed authentication attempts and automatically, temporarily blocks the source — without a human watching logs in real time.

3.2 First Principles

Fail2Ban is not, itself, a firewall, and not itself a security boundary — it's an *orchestration layer* that sits between two things that already exist: a log source it can parse, and a firewall backend it can issue commands to. Its entire function is pattern recognition over text plus automated, time-bounded enforcement.

Filters and regex: turning unstructured text into structured events

A "filter" in Fail2Ban is fundamentally a named regular expression (or set of them) designed to match one specific failure pattern in a specific log format, with a mandatory capture group isolating the source IP address. For SSH, the canonical example matches lines like `Failed password for invalid user admin from 203.0.113.7 port 51422 ssh2`, extracting `203.0.113.7` as the offending host. The regex's job is narrowly scoped: recognize *this exact failure signature*, reliably, without false-matching unrelated log lines that merely happen to contain an IP address for unrelated reasons (a successful login notification, for instance, must never accidentally match a *failure* filter).

Jails: binding a filter to an action and a threshold

A "jail" is the configuration unit that ties a filter to: which log file to watch, how many matches within what time window constitute an attack (`maxretry` and `findtime`), and how long an offending IP should be blocked once that threshold is crossed (`bantime`). The jail is where policy decisions live — the filter just recognizes *that* a failure happened; the jail decides *how many is too many, how fast, and what the consequence is*.

The backend: how Fail2Ban actually watches a log file

Fail2Ban needs a mechanism to detect new log lines as they're written, and it supports several interchangeable backends for this. `pyinotify` and `systemd`'s journal-watching backend use kernel-level file-change notification (`inotify`) or journal change subscription respectively — both efficient, event-driven mechanisms that wake the Fail2Ban process only when new data actually exists, rather than burning CPU on a busy-poll loop. A `polling` backend, by contrast, periodically re-reads the file's current size/position on a timer — functionally equivalent in outcome but materially less efficient, used mainly as a fallback when `inotify` isn't available or when watching filesystems (like some network mounts) where `inotify` semantics don't reliably apply.

How an enforcement decision actually becomes a network-level block

Once a jail's threshold is crossed, Fail2Ban doesn't implement packet filtering itself — it shells out to whatever firewall backend is configured (in this lab, `firewalld`'s rich-rule mechanism, since both Rocky Linux and current Ubuntu Server in this lab's configuration are managed via `firewalld`) and issues a command to insert a block rule for that specific source IP. This is where the orchestration-layer framing matters: Fail2Ban's entire value is deciding *when* and *for how long* to ban, while the actual packet-dropping mechanism is delegated entirely to infrastructure that already exists and is already trusted.

Where the block actually happens in the kernel: Netfilter

`firewalld` itself is a management daemon sitting on top of the kernel's Netfilter framework — the actual packet-filtering engine compiled into the Linux kernel, operating at several defined hook points in the network stack (`PREROUTING`, `INPUT`, `FORWARD`, `OUTPUT`, `POSTROUTING`) where packets can be inspected and a verdict (`ACCEPT`, `DROP`, `REJECT`) applied before the packet proceeds further. When Fail2Ban issues a ban, the resulting rule is ultimately installed into Netfilter's `INPUT` chain processing (via `nftables` rules on current `firewalld` versions, which use `nftables` as its backend rather than legacy `iptables`), meaning the offending packet is evaluated and dropped *before* it ever reaches the SSH daemon at all — the connection attempt is refused at the network layer, not the application layer, so the attacker doesn't even get the satisfaction of a TCP handshake completing, let alone reaching `sshd`'s authentication logic.

'DROP' vs 'REJECT', and why it matters here

A `DROP` verdict silently discards the packet with no response whatsoever; the sending host's connection attempt simply times out with no information about why. A `REJECT` verdict actively sends back a TCP RST or ICMP unreachable, informing the sender immediately that the connection was refused. For Fail2Ban's purpose, `DROP` is generally preferred: it gives an automated scanning tool no immediate signal distinguishing "this host doesn't exist," "this port is closed," and "you have been specifically banned" — denying useful reconnaissance information to whatever is attacking.

3.3 Architecture

Fail2Ban runs locally on `backend01` and `backend02` — the nodes actually exposing SSH to attack surface — watching `sshd` authentication failures via each host's own local journal/log, and enforcing bans via each host's own local `firewalld` instance. This is deliberately *not* centralized through `edge01`: a ban decision needs to take effect as close to the actual point of attack as possible, with minimal latency between detection and enforcement, and each node already has direct, local access to both its own authentication log and its own firewall — adding a network hop to centralize this would only slow down the one thing that benefits most from being fast.

Critically, this lab's Phase 5 logging work (Part 2) means a ban event *itself* gets logged by `sshd`/Fail2Ban and forwarded to `edge01` like any other log line — so while enforcement stays local and fast, *visibility* into enforcement still centralizes naturally through the infrastructure already built, with no additional work required.

3.4 Linux Internals

The full chain from kernel to ban

1. A remote host's failed SSH authentication attempt is processed entirely by `sshd` in userspace — the TCP handshake and SSH protocol negotiation happen normally, since at this point Fail2Ban has not yet made any decision; the attacker is not yet banned.
2. `sshd` logs the failure via the standard syslog API, landing in `journald`'s local journal exactly as described in Part 2.
3. Fail2Ban's journal-backed monitor (using `systemd.journal`'s Python bindings, in this lab's case, given both distributions' journald-centric logging) receives the new entry as soon as it's written — an event-driven wakeup, not a poll.
4. The configured filter's regex is evaluated against the message. A match extracts the offending IP into Fail2Ban's internal tracking state for that jail.
5. Fail2Ban's internal ticket/counter system increments the failure count for that specific IP within the jail's `findtime` window.
6. If the count reaches `maxretry` within that window, Fail2Ban's action executor runs the configured ban command — by default, a `firewall-cmd` invocation (or, more precisely, a direct interaction with `firewalld`'s D-Bus interface for lower-latency control rather than re-invoking the CLI tool's own startup overhead each time) inserting a rich rule that matches the offending source IP and applies a `drop` verdict, scoped to a `fail2ban` zone or rich-rule construct.
7. This rule is translated by `firewalld` into the underlying `nftables` ruleset and loaded into the kernel's Netfilter tables.
8. From this point forward, any packet from that IP arriving at the `INPUT` chain — *before* it can reach the socket `sshd` is listening on — is matched against the new rule and dropped at the kernel level. `sshd` in userspace never even sees the connection attempt; the rejection happens entirely within the kernel's packet-processing path.
9. After `bantime` elapses, Fail2Ban's internal scheduler issues the corresponding unban command, removing the rule and restoring normal reachability for that IP.

This last step matters for understanding state lifecycle: the ban is not permanent infrastructure state — it's a temporary rule Fail2Ban itself is responsible for both creating *and* later removing, tracked entirely in Fail2Ban's own runtime memory (and optionally persisted via its SQLite-backed ban database for survival across a Fail2Ban restart).

3.5 Implementation

Step 1 — Installation (both ``backend01`` and ``backend02``)

```
# Ubuntu (both backend nodes):
sudo apt update
sudo apt install fail2ban -y
```

Step 2 — Jail configuration

Never edit `jail.conf` directly — it's the package-maintained default and gets overwritten on updates. Create `/etc/fail2ban/jail.local`.

Critical distribution-specific detail: `backend01` and `backend02` both run Ubuntu, which uses UFW as its default firewall management layer — not `firewalld`. Setting `banaction = firewallcmd-rich-rules` on Ubuntu means Fail2Ban attempts to invoke `firewall-cmd`, which does not exist on Ubuntu, and the enforcement action silently fails. Fail2Ban's internal state records a ban, but no kernel-level rule is ever inserted — the attacker continues to

connect. Always match `banaction` to the actual firewall management layer present on the specific distribution.

On both `backend01` and `backend02` (Ubuntu — UFW):

```
[DEFAULT]
bantime = 1h
findtime = 10m
maxretry = 5
backend = systemd
banaction = ufw

[sshd]
enabled = true
port = ssh
filter = sshd
maxretry = 5
```

Step 3 — Enable and start

```
sudo systemctl enable --now fail2ban
```

Step 4 — Confirm the jail is active

```
sudo fail2ban-client status sshd
```

3.6 Command Breakdown

<code>fail2ban-client status</code>	Queries the running daemon for overall jail summary
<code>fail2ban-client status sshd</code>	Queries detailed state of one specific jail: current/total failed, current/total banned, and the banned IP list
<code>fail2ban-client set sshd unbanip <ip></code>	Manually forces an immediate unban, bypassing the <code>bantime</code> countdown — used operationally when a ban is confirmed to be a false positive
<code>fail2ban-client set sshd banip <ip></code>	Manually forces an immediate ban — used for proactive blocking of a known-bad IP outside the normal detection pipeline
<code>fail2ban-regex <logfile> <filterfile></code>	A dry-run tool: tests whether a given filter's regex actually matches lines in a real log file, without taking any enforcement action — the correct way to validate a custom filter before trusting it in production

3.7 Configuration Breakdown

- **findtime = 10m** — the rolling time window within which failures are counted toward the threshold. A **maxretry** of 5 within a 10-minute window is a meaningfully different policy than 5 failures ever — the window is what distinguishes "an attacker hammering the service" from "a legitimate user who mistyped their key passphrase a few times over several unrelated days."
- **bantime = 1h** — how long an offending IP remains blocked once banned. Short enough that a legitimate user who triggers it accidentally isn't locked out indefinitely; long enough to meaningfully disrupt an automated scanning tool's economics.
- **backend = systemd** — explicitly selects the journal-based, event-driven log-watching mechanism described in 3.4, appropriate given both nodes' journald-centric logging.
- **banaction = ufw** — selects UFW as the enforcement backend, which is the correct choice on Ubuntu. Using **firewallcmd-rich-rules** on Ubuntu would silently fail at every ban attempt because **firewall-cmd** does not exist on that distribution. The **banaction** must always match the actual firewall management layer installed on the host, not an assumed default. Verify enforcement independently of Fail2Ban's own status output by checking **sudo ufw status numbered** for the expected deny rule.

3.8 Packet Flow

For a connection attempt arriving *after* a ban is already active:

1. An external host with a banned source IP sends a TCP SYN packet toward **backend02**'s SSH port.
2. The packet arrives at the physical NIC and enters the kernel's network stack.
3. Netfilter's **PREROUTING/INPUT** chain processing evaluates the packet against the loaded ruleset, which now includes the rich rule Fail2Ban inserted.
4. The rule matches on source IP and applies a **drop** verdict.
5. The packet is discarded inside the kernel. It never reaches the socket layer; **sshd**'s listening socket never sees a connection attempt; no SYN-ACK is ever sent.
6. The remote attacker's TCP stack simply times out waiting for a response it will never receive.

This is the architecturally important distinction from application-layer rejection: the cost of handling a banned attacker, post-ban, is essentially zero — a single rule-table lookup in the kernel, versus a full TCP handshake plus SSH protocol negotiation plus authentication-failure processing if the connection had been allowed to reach **sshd** at all.

3.9 Process Flow

Fail2Ban runs as a long-lived daemon (**fail2ban-server**, controlled via the **fail2ban-client** CLI as a thin client/server pair — itself a deliberate design choice letting configuration be reloaded or queried without restarting the actual monitoring process and losing in-memory ban state). At startup, it reads **jail.local**, instantiates a monitoring thread per enabled jail, and for **systemd**-backend jails, opens a journal reader filtered to the relevant unit (**sshd.service**). Each new matching journal entry triggers filter evaluation; matches update per-IP failure counters held in memory; counters that cross threshold trigger the configured ban action; a separate internal scheduler thread tracks each active ban's expiry and issues the unban action once **bantime** elapses.

3.10 Verification

```
sudo fail2ban-client status sshd
```

On **backend02**, do not be surprised if real external IPs are already appearing in the banned list before any deliberate test is run. **backend02** has a public Azure IP and automated credential-stuffing scanners begin probing the moment a server becomes internet-reachable — Fail2Ban detecting and banning real attackers before any test is initiated confirms the jail is working in production conditions, not just a lab scenario.

Testing on backend02 (internet-facing): run the deliberate test from a source IP you do not need SSH access from. Six iterations guarantees crossing **maxretry = 5**:

```
# Never run from a session you need to keep – your source IP will be banned:  
for i in 1 2 3 4 5 6; do ssh wronguser@backend02 -p 22; done
```

```
sudo fail2ban-client status sshd  
sudo ufw status numbered
```

ufw status numbered confirms enforcement at the kernel level. If **fail2ban-client status** shows a ban but **ufw status** shows no corresponding deny rule, the **banaction** is wrong — the most common cause is **firewallcmd-rich-rules** on Ubuntu where **firewall-cmd** does not exist.

Testing on backend01 (private network — different approach required): **backend01** is on a private Host-only network and has no exposure to external attackers. The only hosts that can reach its SSH port are **edge01** and the hypervisor. Run the test from **edge01**:

```
# On edge01:  
for i in 1 2 3 4 5 6; do ssh wronguser@192.168.56.11 -p 22; done
```

Then verify on **backend01**:

```
sudo fail2ban-client status sshd  
sudo ufw status numbered
```

192.168.56.10 (edge01's Host-only IP) should appear in the banned list — demonstrating that Fail2Ban enforces on behavior alone, not on network trust. A compromised **edge01** hammering **backend01**'s SSH gets banned as fast as an external attacker hitting **backend02**.

3.11 Failure Simulation

Break 1 — Wrong `logpath` for the distribution.

Ubuntu's **sshd** log path conventions can differ depending on whether rsyslog or pure journald is the system of record; an incorrectly assumed path leaves the jail "enabled" but never actually receiving data.

Symptom: **fail2ban-client status sshd** perpetually shows zero failed attempts, even during an active, ongoing brute-force attempt visible in **journalctl -u sshd** directly.

Diagnosis: **fail2ban-regex <correct-source> sshd** confirms whether the filter *would* match real log content — if it matches the journal output directly but the jail isn't catching it, the configured backend/logpath, not the regex, is the problem.

Fix: confirm `backend = systemd` is set explicitly (rather than `auto`, which can resolve incorrectly depending on what log files happen to exist on disk) so Fail2Ban reads from the journal directly rather than guessing at a flat-file path that may not be populated.

Break 2 — `firewalld` not running, or running but `banaction` referencing a backend not installed.

Symptom: Fail2Ban's logs (`journalctl -u fail2ban`) show it detected the threshold being crossed and attempted to execute the ban action, but the action itself errors out.

Root cause: Fail2Ban's detection layer and its enforcement layer are independently failable — detecting correctly is no guarantee enforcement succeeded, because they're separate subsystems connected only by a shell command.

Diagnosis: `systemctl status firewalld` confirms whether the underlying enforcement mechanism is even available to be commanded.

Fix: ensure `firewalld` is enabled and running before Fail2Ban attempts to depend on it; consider an explicit `systemd` unit dependency (`After=firewalld.service`, `Requires=firewalld.service`) so Fail2Ban doesn't start in a state where its enforcement backend isn't yet ready.

Break 3 — False positives from a legitimate but misbehaving client.

Symptom: a legitimate administrator gets banned after a string of genuine, accidental authentication failures (e.g., an old SSH key still cached locally being offered before the correct one).

Diagnosis: `fail2ban-client status sshd` confirms the IP and ban reason; cross-referencing the centralized log copy on `edge01` (Part 2) for that timeframe confirms whether the failures were genuine accidental attempts versus an actual attack pattern that happened to originate from a shared/NATed IP a legitimate user also uses.

Fix, immediate: `fail2ban-client set sshd unbanip <ip>`.

Fix, structural: consider a higher `maxretry` or a `findtime/maxretry` ratio tuned against this lab's actual observed legitimate-user behavior, and consider an explicit `ignoreip` allowlist for known-trusted administrative source ranges.

3.12 Troubleshooting

The diagnostic order here separates three independently-failable layers: is the *detection* working (does the filter match real log content, verified via `fail2ban-regex`)? Is the *counting/threshold* logic working as configured (does `fail2ban-client status` show counters incrementing during a real test)? Is the *enforcement* working (does `firewall-cmd --list-rich-rules` actually reflect the ban, and does a packet test from the banned IP confirm it's actually dropped at the kernel level, not just recorded in Fail2Ban's own state)? A surprising number of real-world Fail2Ban issues are enforcement-layer failures masquerading as detection-layer failures, because Fail2Ban's own status output can show a "ban" was recorded in its internal state even if the underlying `firewalld` command silently failed.

3.13 Production Engineering Notes

- **Allowlisting:** production deployments always configure `ignoreip` with known-trusted source ranges (office IPs, VPN egress ranges, monitoring infrastructure) to eliminate the

operational risk of a legitimate automated system (a monitoring probe doing repeated health checks, for instance) ever being banned by its own infrastructure.

- **Escalating ban times:** Fail2Ban supports `bantime.increment`, multiplying the ban duration for repeat offenders rather than applying a flat duration every time — meaningfully raising the cost for a persistent attacker without raising the cost imposed on a one-time accidental trigger.
- **Cross-host correlation:** this lab's two backend nodes run Fail2Ban independently, with no shared state — an IP banned on `backend02` is not automatically banned on `backend01`. A more mature deployment would centralize ban *decisions* (not just visibility) across nodes, often via a shared `ipset` synchronized through configuration management or a dedicated coordination layer, so an attack pattern detected on one node immediately protects the other.
- **Interaction with the WireGuard overlay:** Fail2Ban's SSH jails apply to each node's public-facing SSH exposure; the overlay's own `AllowedIPs`-based access control (Part 1) is an entirely separate, cryptographic enforcement layer that doesn't need Fail2Ban at all — WireGuard simply never accepts traffic from anything not already on the configured allowlist, which is a stronger guarantee than rate-limited banning for that specific path.

3.14 Interview Reference

Q: Is Fail2Ban a firewall?

A: No — it's a log-driven decision engine that issues commands to an existing firewall backend (here, `firewalld`/Netfilter). It contributes the "when and for how long" policy logic; the actual packet-dropping mechanism is delegated entirely to infrastructure that already exists independently of Fail2Ban.

Q: Why does a Fail2Ban-enforced block happen at the kernel level rather than inside `sshd` itself?

A: Because the ban is implemented as a Netfilter rule evaluated during `INPUT` chain processing, before the kernel ever hands the packet to the socket `sshd` is listening on. This means a banned attacker's connection attempt is rejected (or silently dropped) before the SSH daemon expends any resources on the TCP handshake or protocol negotiation at all — meaningfully cheaper than rejecting at the application layer, and more resistant to resource-exhaustion attempts from a high volume of banned-but-still-attempting sources.

Q: A jail shows zero detections despite an obviously ongoing attack visible in the raw logs. What do you check first?

A: Whether the filter's regex actually matches the real log line format being produced — verified directly with `fail2ban-regex` against the actual log source — before assuming anything about thresholds or enforcement, since a non-matching filter means nothing downstream of detection ever runs at all.

Part 4 — rsync: Disaster Recovery

4.1 Engineering Problem

None of the infrastructure built across Phases 1 through 5 has a recovery story. If `backend01`'s disk fails, or `backend02`'s Azure VM is deleted accidentally or otherwise, there is currently no path back to a known-good state short of rebuilding from scratch and re-running every prior phase's manual steps by hand. We need a backup mechanism that is efficient enough to run frequently, reliable enough to trust for actual disaster recovery, and respectful enough of bandwidth that running it across the Azure WAN link doesn't itself become an operational burden.

This sounds straightforward until the implementation is attempted for real — at which point several assumptions embedded in the original design collapse against production constraints. This section documents both the correct final implementation and the exact failure modes that drove each design decision, because understanding why a particular approach fails is as valuable as knowing which approach works.

4.2 First Principles

Why not `scp`

`scp` is a complete-file-copy tool: every invocation transfers the entirety of every requested file, regardless of whether the destination already has an identical or near-identical copy from a previous run. For a *repeated*, scheduled backup of mostly-unchanged data — which is precisely what disaster recovery backups are — `scp` would mean retransmitting gigabytes of unchanged data every single run, purely because it has no concept of "what's already there" at all. Given this lab's WAN link characteristics (already documented as latency- and reliability-sensitive in Phase 4), that's not just wasteful, it's actively the wrong tool for a link where every byte transferred has a real cost.

The delta-transfer algorithm: rsync's actual innovation

`rsync`'s defining contribution is the *rsync algorithm* — a method for synchronizing two files across a network while transmitting only the *differences* between them, even when neither side knows in advance exactly what changed. The receiving side divides its existing copy of a file into fixed-size blocks and computes a checksum for each block using a fast, rolling checksum function. These checksums — small relative to the actual block data — are sent to the sending side. The sender slides a window across its own version of the file, looking for checksum matches at every possible byte offset, not just block boundaries — this is what "rolling" means: the checksum recomputes incrementally as the window advances one byte at a time rather than from scratch, making the scan computationally tractable across large files.

Matching regions are referenced rather than retransmitted. Non-matching bytes are sent as literal data. The receiver reconstructs the new file by stitching together what it received with what it already had. The practical consequence: appending a few lines to a large config file results in a transfer proportional to the *change*, not the file — a property `scp` cannot offer because it treats the file as an opaque, indivisible blob regardless of how much is identical to what already exists at the destination.

Transport: rsync over SSH

rsync defines the synchronization algorithm; it doesn't define how bytes physically move between hosts. In this lab, rsync runs over SSH, meaning every backup transfer inherits the encryption, authentication, and access control already established for SSH in earlier phases. From the network's perspective, the entire rsync exchange is indistinguishable from any other SSH session — no separate port, no separate protocol.

Archive mode and metadata preservation

A backup that restores file contents correctly but loses ownership, permissions, or timestamps is not a faithful backup — restoring a configuration file with wrong ownership can silently break the service depending on it. rsync's **-a** (archive) flag bundles: **-r** (recurse), **-l** (preserve symlinks), **-p** (preserve permissions), **-t** (preserve modification times), **-g** (preserve group), **-o** (preserve owner), and **-D** (preserve device files). Using **-a** is both more concise and less error-prone than specifying each individually.

Why personal interactive keys break unattended backup

This is the production failure that drove the most significant architectural change from the original design. The initial plan used `/home/eudofia/.ssh/id_ed25519` — the personal ED25519 key used interactively throughout the lab. This key is passphrase-protected, which is correct security practice for an interactive key, but fatal for a background daemon: a systemd service has no interactive terminal to prompt for the passphrase, so rsync silently fails at the authentication step every single unattended run, with no obvious error visible unless the logs are inspected deliberately.

The fix is a dedicated machine-to-machine (M2M) key with **no passphrase**, owned exclusively by root, used only for this automation purpose. The absence of a passphrase is not a security regression here — it is offset by: strict file permissions (`600`, owned by root), the key being authorized only on the specific remote user accounts needed for backup, and the key being usable only for SSH/rsync operations, not interactive login to `edge01`.

Why remote sudo elevation is required

Even with a valid SSH connection, pulling `/etc/` as an unprivileged user fails on files that are root-readable-only — notably `/etc/shadow` (the password hash file), `/etc/gshadow`, and systemd credential store directories. rsync exits with code 23 (partial transfer) or generates **Permission denied (13)** errors on these files. The fix is `--rsync-path="sudo rsync"`, which instructs SSH to launch the remote rsync process as root rather than as the logged-in user. This requires a specific, minimal sudoers entry on each backend node authorizing the backup user to run rsync without a password prompt — unattended, again, so interactive sudo is not an option.

Why Azure breaks `ssh-copy-id`

The standard method for distributing a new public key to a remote host is `ssh-copy-id`, which works by SSHing in with password authentication and appending the new key. Azure Ubuntu VMs disable password authentication by default at the cloud-infrastructure level, not just in `sshd_config` — meaning `ssh-copy-id` has no fallback authentication mechanism available. The solution is to use an *already-authorized* key to open the SSH connection and pipe the new public key through it, bypassing the password requirement entirely.

Exit code 24: the harmless warning rsync gets wrong

rsync exit code 24 means "some files vanished during the transfer." This sounds alarming but is almost always harmless in a live `/etc/` backup: transient lock files (`.pwd.lock`, for example) may exist when rsync begins scanning but be removed by the OS before rsync transfers them. This is expected behavior on any live system. Without explicit handling, a naive script treating any non-zero exit code as failure will report these runs as failed — generating spurious failure alerts on every single backup run, which trains engineers to ignore them, which eventually means a real failure is also ignored. Exit code 24 must be explicitly caught and treated as success.

4.3 Architecture

`edge01` acts as the backup destination and initiator — it pulls from `backend01` and `backend02` rather than each backend pushing to it, keeping trigger logic centralized on the control node. A dedicated root-owned M2M SSH key on `edge01` authenticates to each backend node. Remote rsync processes run with sudo elevation to access privileged files. Backup data lands in `/backup/<nodename>/etc/` on `edge01`, owned by root, created automatically by the script if absent.

`backend01`'s backup traffic moves over the Host-only link (`192.168.56.11`). `backend02`'s moves over the WireGuard overlay (`10.100.0.2`) — same routing constraint established for rsyslog in Part 2, applied identically here.

4.4 Linux Internals

When rsync runs over SSH with `--rsync-path="sudo rsync"`, the sequence on the remote side is: SSH daemon accepts the connection, authenticates the M2M key, executes the requested command — which is `sudo rsync --server ...` rather than the bare `rsync --server` a normal rsync invocation would use. The sudoers entry on the backend node authorizes this specific command without requiring a password, so the elevation happens silently and automatically. From that point, the remote rsync process runs as root, with full read access to the entire filesystem including privileged files.

On the filesystem side, rsync writes each transferred file to a temporary name first, then performs an atomic rename once the file transfer completes. This crash-safety property means an interrupted transfer never leaves a half-written file under its final name — the previous complete version remains intact until a new transfer fully succeeds. The `-z` compression flag operates at the SSH transport layer, compressing the delta data stream before it crosses the network — particularly effective for the plain-text configuration files that dominate `/etc/`.

4.5 Implementation

Implementation of disaster recovery follows two sequential phases: first, manual validation confirming that rsync can reach and read from each backend node over the correct network path; second, hardening for unattended automation by replacing the personal interactive key with a dedicated machine-to-machine key and elevating remote privileges to read root-owned files. Never automate something you haven't first validated manually — a scheduled backup job that has never been confirmed to actually work is an unverified assumption, not a recovery capability.

Phase A — Manual Validation (run first, before any automation)

Step A1 — Create the backup destination directories on `edge01`

```
sudo mkdir -p /backup/backend01/etc
sudo mkdir -p /backup/backend02/etc
sudo chown -R eudofia:eudofia /backup
```

mkdir -p creates the entire path including any missing intermediate directories. The **chown** step is essential: without it, the backup destination is owned by root, and running rsync without **sudo** (appropriate for initial testing) will fail immediately with a permission error writing the first file. Changing ownership to **eudofia** allows rsync invoked as the normal user to write to the destination.

Step A2 — Manual rsync from `edge01` to `backend01` (Host-only path)

```
rsync -avz --delete \
  --exclude='/credstore' \
  --exclude='/credstore.encrypted' \
  -e "ssh -i /home/eudofia/.ssh/id_ed25519" \
  eudofia@192.168.56.11:/etc/ \
  /backup/backend01/etc/
```

The **--exclude** flags are required even in the manual test. Without them, rsync will attempt to open **/etc/credstore** and **/etc/credstore.encrypted** — directories that systemd creates as root-only credential stores. As an unprivileged user, rsync cannot read them and exits with **Permission denied (13)** errors, which on some rsync versions also cause a non-zero exit code that would be misread as a backup failure.

What to expect: rsync will output a file list during transfer, ending with a summary line showing bytes transferred and speed. The first run transfers everything; subsequent runs will show much smaller transfers because the delta algorithm only moves what changed.

Step A3 — Manual rsync from `edge01` to `backend02` (WireGuard overlay path)

```
rsync -avz --delete \
  --exclude='/credstore' \
  --exclude='/credstore.encrypted' \
  -e "ssh -i /home/eudofia/.ssh/id_ed25519" \
  ubuntu@10.100.0.2:/etc/ \
  /backup/backend02/etc/
```

Note the username is **ubuntu** (Azure Ubuntu default) and the target IP is **10.100.0.2** — **backend02**'s WireGuard overlay address, not its public IP. Traffic routes through the encrypted tunnel, not directly to **158.158.37.102**, for the same routing reasons that drive rsyslog to use **10.100.0.1** as its destination.

Step A4 — Verify backup contents and metadata

```
ls /backup/backend01/etc/ | head -20
ls /backup/backend02/etc/ | head -20
```

You should see familiar **/etc/** content: **ssh/**, **passwd**, **hostname**, **fstab**, **rsyslog.conf**, **fail2ban/**, **wireguard/**. Confirm metadata fidelity:

```
stat /backup/backend01/etc/ssh/sshd_config
```

Compare the permissions and timestamps against the source. If they match, **-a** (archive mode) worked correctly.

Why manual validation must happen before automation: if the manual rsync test fails, there is a resolvable problem (connectivity, authentication, permissions) that must be understood and

fixed before it can be hidden inside an automated job. Automating a broken process does not fix the process — it only makes the breakage harder to see.

Phase B — Hardened Automation (after Phase A validates connectivity)

The manual approach using the personal interactive key (`/home/eudofia/.ssh/id_ed25519`) cannot be used in automation for two reasons discovered during implementation: the key is passphrase-protected (a background daemon has no terminal to prompt), and the remote user lacks privileges to read root-owned files in `/etc/` (notably `/etc/shadow` and `/etc/gshadow`). Phase B addresses both.

Step B1 — Generate the dedicated M2M automation key on `edge01` (as root)

```
sudo ssh-keygen -t ed25519 \
  -f /root/.ssh/id_ed25519_automation \
  -C "systemd-backup-automation" \
  -N ""
```

`-N ""` sets an empty passphrase explicitly — this is intentional and required for unattended use. The key lives under `/root/.ssh/` with permissions only root can read. Never use this key for interactive human SSH sessions; it has a single, narrow, auditable purpose.

Step B2 — Distribute the automation public key to backend nodes

Standard `ssh-copy-id` fails against Azure Ubuntu nodes because Azure disables password authentication at the infrastructure level — not just in `sshd_config`, but at the cloud fabric layer — leaving `ssh-copy-id` with no fallback authentication mechanism. The solution is to pipe the new public key through an existing authorized interactive SSH session:

```
# Push to backend02 (Azure) via existing interactive key:
sudo cat /root/.ssh/id_ed25519_automation.pub | \
  ssh -i /home/eudofia/.ssh/id_ed25519 ubuntu@10.100.0.2 \
  'cat >> ~/.ssh/authorized_keys'

# Push to backend01 (local VirtualBox) via existing interactive key:
sudo cat /root/.ssh/id_ed25519_automation.pub | \
  ssh -i /home/eudofia/.ssh/id_ed25519 eudofia@192.168.56.11 \
  'cat >> ~/.ssh/authorized_keys'
```

This uses the already-authorized personal key to open the SSH tunnel, appends the automation public key to `authorized_keys` on the remote host, and closes. No password prompt is ever required because the tunnel authenticates with the existing key.

Verify the key was distributed correctly:

```
ssh -i /root/.ssh/id_ed25519_automation -o BatchMode=yes ubuntu@10.100.0.2 echo
"automation key works"
ssh -i /root/.ssh/id_ed25519_automation -o BatchMode=yes eudofia@192.168.56.11 echo
"automation key works"
```

Both should print "automation key works" immediately with no passphrase prompt. If either hangs or fails, the key was not correctly appended to `authorized_keys` on that backend.

Step B3 — Configure passwordless sudo for rsync on each backend node

`--rsync-path="sudo rsync"` causes the remote rsync to run as root, enabling it to read `/etc/shadow` and other privileged files. This requires the connecting user to have passwordless sudo specifically for rsync — not blanket passwordless sudo, which would be an excessive privilege grant.

On **backend01**:

```
echo "eudofia ALL=(root) NOPASSWD: /usr/bin/rsync" | sudo tee /etc/sudoers.d/rsync-backup
sudo chmod 440 /etc/sudoers.d/rsync-backup
```

On **backend02**:

```
echo "ubuntu ALL=(root) NOPASSWD: /usr/bin/rsync" | sudo tee /etc/sudoers.d/rsync-backup
sudo chmod 440 /etc/sudoers.d/rsync-backup
```

Validate the entry before depending on it:

```
sudo visudo -c -f /etc/sudoers.d/rsync-backup
```

Then confirm the elevation actually works end-to-end:

```
# From edge01, test that remote sudo rsync is authorized:
ssh -i /root/.ssh/id_ed25519_automation -o BatchMode=yes ubuntu@10.100.0.2 "sudo rsync
--version"
ssh -i /root/.ssh/id_ed25519_automation -o BatchMode=yes eudofia@192.168.56.11 "sudo
rsync --version"
```

Both should print rsync's version string. If either produces a sudo password prompt (which **BatchMode=yes** will refuse, causing immediate failure), the sudoers entry is missing or has a syntax error.

Step B4 — Create the hardened backup script on `edge01`

/usr/local/sbin/edgesecure-backup.sh:

```
#!/usr/bin/env bash
set -euo pipefail

LOGTAG="edgesecure-backup"
KEY="/root/.ssh/id_ed25519_automation"

run_backup() {
    local label="$1"
    local target="$2"
    local dest="$3"
    local rc=0

    /usr/bin/rsync -az --delete \
        --rsync-path="sudo rsync" \
        --exclude='/credstore' \
        --exclude='/credstore.encrypted' \
        -e "ssh -i ${KEY} -o BatchMode=yes -o ConnectTimeout=10" \
        "${target}" "${dest}" || rc=$?

    if [[ ${rc} -eq 0 ]]; then
        logger -t "${LOGTAG}" "SUCCESS: ${label} backup completed cleanly."
    elif [[ ${rc} -eq 24 ]]; then
        logger -t "${LOGTAG}" "SUCCESS: ${label} backup completed (ignored vanished transient
files)."
    else
        logger -t "${LOGTAG}" "FAILURE: ${label} backup failed with exit code ${rc}"
    fi
}

mkdir -p "/backup/backend01/etc/" "/backup/backend02/etc/"
```

```
run_backup "backend01" "eudofia@192.168.56.11:/etc/" "/backup/backend01/etc/"
run_backup "backend02" "ubuntu@10.100.0.2:/etc/"      "/backup/backend02/etc/"

sudo chown root:root /usr/local/sbin/edgesecure-backup.sh
sudo chmod 700 /usr/local/sbin/edgesecure-backup.sh
```

chmod 700 rather than **750**: the script holds the path to an unencrypted key and runs as root. No non-root user should be able to execute or read it.

4.6 Command Breakdown

-a	Archive mode — bundles recursive copy, permission/ownership/timestamp/symlink/device-file preservation
-z	Compress data stream in transit
--delete	Removes destination files absent from source — keeps backup a true mirror
--rsync-path="sudo rsync"	Launches remote rsync process as root via sudo, granting read access to privileged files
--exclude=	Skips directories that are structurally unreadable even by root (systemd credential stores)
-o BatchMode=yes	Disables any interactive prompt in the SSH layer — if authentication can't complete silently, it fails immediately with an error rather than hanging
-o ConnectTimeout=10	Limits SSH connection establishment to 10 seconds — prevents the script from hanging indefinitely on an unreachable host
rc=\$? (on rsync failure)	Captures rsync's exit code without triggering set -e 's immediate-abort, allowing the script's own exit code evaluation to run

4.7 Configuration Breakdown

- **-N ""** in key generation — explicitly sets an empty passphrase; without this, **ssh-keygen** prompts interactively, which breaks scripted key generation.
- **/etc/sudoers.d/rsync-backup** — placing the sudoers entry in a drop-in file rather than directly editing **/etc/sudoers** follows the same upgrade-safety principle as **/etc/rsyslog.d/**: package updates don't overwrite drop-in files.
- **`chmod 700` on the script** — tighter than the original **750**; since the script holds the path to an unencrypted key and runs as root, no non-root user should be able to execute or read it.

- **Exit code 24 treated as success** — explicitly, deliberately, with a distinct log message noting what happened. This is not silently ignoring errors; it is categorizing a known-harmless condition correctly so that real failures (any other non-zero exit code) remain unambiguously visible.

4.8 Packet Flow

Identical to the general rsync-over-SSH packet flow, with one addition: the SSH session's remote command is `sudo rsync --server ...` rather than `rsync --server ....` From the network's perspective this is indistinguishable — the sudo invocation happens entirely within the remote host's process space, invisible to any network observer. For `backend02`, the SSH session itself is carried over the WireGuard tunnel exactly as traced in Part 1, Section 1.8 — the rsync protocol runs inside the SSH channel, which runs inside the WireGuard encrypted UDP datagram, which crosses the public internet as opaque encrypted traffic.

4.9 Process Flow

1. systemd timer fires at the scheduled time (Part 5 covers this in detail).
2. systemd starts `edgesecure-backup.service`, which executes the script as root.
3. The script's `run_backup` function is called for `backend01` first.
4. `/usr/bin/rsync` is invoked with the M2M key; SSH connects to the backend node and launches `sudo rsync --server` on the remote side.
5. The two rsync processes exchange checksums, negotiate the delta, and transfer only changed blocks.
6. rsync exits; the exit code is captured into `rc`.
7. The appropriate `logger` call fires, writing a structured success or failure line into the local journal — which Part 2's rsyslog pipeline picks up and forwards to `edge01`'s own centralized log store automatically, with no additional wiring needed.
8. The function is called again for `backend02` over the WireGuard overlay.
9. The script exits; systemd records the service run's outcome in the journal.

4.10 Verification

Manual validation (Phase A):

```
# Dry run first – reports what would transfer without writing anything:
rsync -avzn --delete \
  --exclude='/credstore' \
  --exclude='/credstore.encrypted' \
  -e "ssh -i /home/eudofia/.ssh/id_ed25519" \
  eudofia@192.168.56.11:/etc/ \
  /backup/backend01/etc/
```

The `-n` flag performs the full checksum comparison and reports exactly what *would* transfer without actually writing anything. Run this before every manual rsync that involves `--delete`, since `--delete` will remove files from the destination that no longer exist on the source — confirming scope before writing prevents accidents.

Automated validation (Phase B):

```
# Manual trigger of the automated script (same environment systemd will use):
sudo /usr/local/sbin/edgesecure-backup.sh
```

```
# Check journal output:
journalctl -t edgesecure-backup --since "5 minutes ago"
```

Expected journal output:

```
SUCCESS: backend01 backup completed cleanly.
SUCCESS: backend02 backup completed cleanly.
```

or, if transient lock files vanished during the scan:

```
SUCCESS: backend02 backup completed (ignored vanished transient files).
```

A **FAILURE**: line with a non-zero exit code means a real problem — investigate immediately rather than dismissing it, because the exit code 24 case is already explicitly handled and would not produce a **FAILURE** line.

Confirm backup contents on edge01:

```
ls /backup/backend01/etc/ | head -20
ls /backup/backend02/etc/ | head -20
```

4.11 Failure Simulation

Break 0 — Initial failures during Phase A manual validation (these drove the hardened design).

When first attempting rsync with a bare command, several failures occurred in sequence, each pointing at a different layer:

- **/backup/backend01/etc` failed: No such file or directory** — the destination directory didn't exist. Fix: `sudo mkdir -p /backup/backend01/etc /backup/backend02/etc && sudo chown -R eudofia:eudofia /backup`.
- **Permission denied (13)` on `/etc/credstore` and `/etc/credstore.encrypted** — systemd credential store directories readable only by root. Fix: `--exclude='/credstore' --exclude='/credstore.encrypted'`. These files are runtime credential material managed entirely by systemd — their absence from a `/etc/` backup does not affect recoverability.
- **sudo: 3 incorrect password attempts** then hanging on passphrase — running rsync with `sudo` in an interactive session that also needed the SSH key passphrase created a compounding credential prompt problem. Root cause: the personal interactive key is passphrase-protected, and the script was trying to use `sudo rsync` (needing the sudo password) simultaneously. Fix: Phase B's M2M key design eliminates both problems — no passphrase, no sudo password needed.

Break 1 — M2M key not authorized on the remote host.

Symptom: rsync fails immediately with SSH authentication error; `rc` is 255 (SSH's own connection-failure exit code); journal shows **FAILURE: backend02 backup failed with exit code 255**.

Diagnosis: `ssh -i /root/.ssh/id_ed25519_automation -o BatchMode=yes ubuntu@10.100.0.2 echo test` — reproduces the failure in isolation without rsync, cleanly separating the SSH authentication layer from the rsync protocol layer.

Fix: confirm the automation public key was actually appended to the remote user's `authorized_keys` and that the file has permissions `600`, owned by the correct user.

Break 2 — sudoers entry missing on the remote node.

Symptom: SSH authentication succeeds; rsync connects; the remote `sudo rsync` invocation fails because the backup user isn't authorized for passwordless rsync elevation. rsync exits with code 23 or the SSH session fails with a sudo prompt that `BatchMode=yes` refuses to satisfy.

Diagnosis: `ssh -i /root/.ssh/id_ed25519_automation ubuntu@10.100.0.2 'sudo rsync --version'` — if this prompts for a password or fails, the sudoers entry is missing or has a syntax error. `visudo -c -f /etc/sudoers.d/rsync-backup` confirms syntax validity.

Fix: confirm `/etc/sudoers.d/rsync-backup` exists on the backend node with the correct content.

Break 3 — Exit code 24 misclassified as failure.

Symptom: every backup run logs `FAILURE` despite the backup data being complete and correct; the failure exit code is consistently 24.

Root cause: absence of exit code 24 handling — transient lock files (`.pwd.lock`) exist when rsync begins scanning but are removed before rsync reads them. This is expected behavior on any live system, not a backup failure.

Diagnosis: add `echo "rc=${rc}"` temporarily to the script to capture the actual exit code.

Fix: the explicit `elif [[ ${rc} -eq 24 ]]` branch in the hardened script handles this correctly. rsync exit codes are a structured vocabulary — treating them as binary success/failure produces false alerts that eventually train engineers to ignore all alerts.

4.12 Troubleshooting

Work outward to inward in strict order: can `edge01` reach the backend host at all (plain SSH with the M2M key, `BatchMode=yes`)? Can the remote user execute `sudo rsync` specifically (`sudo rsync --version` via SSH)? Does a dry-run rsync (`-n` flag) complete without errors? Does a real run complete with exit code 0 or 24? Is the resulting data on `edge01` actually faithful (metadata intact, not just file contents)? Each layer's failure is distinct and diagnosed independently — conflating them wastes diagnostic time.

4.13 Production Engineering Notes

- **Key rotation:** the M2M key has no passphrase and lives on disk — its security depends entirely on filesystem permissions and physical/logical access controls on `edge01`. Production environments typically rotate these keys on a defined schedule via configuration management, and immediately on any suspicion of `edge01` compromise.
- **Scope of backup:** this lab backs up `/etc/` only — the configuration layer. A complete disaster recovery posture would also capture application data, home directories, and any stateful data outside `/etc/`. `/etc/` is the correct starting point because it's the layer that takes the most effort to reconstruct by hand and is least likely to be captured elsewhere.
- **Retention:** a mirror backup with `--delete` reflects the current source state. A source-side accidental deletion is mirrored on the next run, destroying the only backup copy. Production deployments layer point-in-time snapshots (via `rsync --link-dest`

rotating backups) on top of the mirror to retain recovery points across accidental deletions.

- **`--delete` risk awareness:** `--delete` is the right choice for a current-state mirror but must be understood as "the backup converges to the source, including deletions." Any engineer running this manually must confirm they are not about to mirror a broken or partially-deleted source state onto the backup.

4.14 Interview Reference

Q: How does rsync transfer less data than a full copy without knowing in advance what changed?

A: The receiving side computes rolling checksums over fixed-size blocks of its existing file and sends those checksums to the sender. The sender slides a window across its own version, checking for matches at every byte offset — not just block boundaries — so it can identify blocks the receiver already has even if content has shifted position due to insertions or deletions. Only genuinely new or changed bytes transmit as literal data; everything else is a short reference saying "you already have this block."

Q: Why does the backup use a dedicated unencrypted key rather than the personal interactive key?

A: A passphrase-protected key requires an interactive terminal to prompt for the passphrase at use time. A background systemd service has no such terminal — authentication silently fails every unattended run. The dedicated M2M key trades passphrase protection for unattended operability, with the security offset coming from strict filesystem permissions, narrow sudoers authorization, and the key being authorized only for the specific remote operations the backup requires.

Q: What does rsync exit code 24 mean and why must it be handled explicitly?

A: Exit code 24 means some files vanished during the transfer — typically transient lock files that existed when rsync began scanning but were removed by the OS before rsync transferred them. This is expected behavior on any live system. Without explicit handling, it registers as a failure, generating spurious alerts on every backup run. Explicitly catching code 24 and treating it as success (with a distinct log message noting the condition) keeps the alerting signal clean so real failures remain unambiguously visible.

Part 5 — systemd Timers: Scheduled Automation

5.1 Engineering Problem

The backup script from Part 4 works correctly when invoked manually. But a backup that runs only when an engineer remembers to trigger it isn't a backup strategy — it's a hope. We need reliable, unattended scheduling that fires even if no one is logged in, that waits for the network to be available before attempting connections, that handles the case where a run was missed while the system was powered off, and that makes its own outcome visible through the same centralized logging pipeline already built in Part 2.

The original design for this section specified `cron` as the scheduling mechanism. Implementation revealed that `cron` satisfies some of these requirements but not all, and that `systemd` timers satisfy all of them more naturally given the rest of this lab's existing `systemd`-centric toolchain. This section documents both, explaining the specific failure modes that make `systemd` timers the correct choice for this workload, while preserving `cron` knowledge as it remains universally relevant to Linux administration.

5.2 First Principles

What cron actually is and where it falls short here

`cron` is a long-running daemon that wakes once per minute, checks every configured schedule against the current wall-clock time, and forks a child process for any job whose schedule matches. It is universal — present on virtually every Linux distribution, understood by virtually every Linux engineer — and its five-field syntax (`minute hour day month weekday`) is one of the most widely recognized scheduling conventions in the Unix tradition.

Its limitations for this specific workload are three:

First, **no dependency awareness**: `cron` fires a job at the scheduled time regardless of whether the system's network is actually available. A backup job that runs at 2:00 AM on a system whose WireGuard tunnel isn't yet fully established simply fails, silently, because `cron` has no mechanism to say "wait until `network-online.target` is reached before running this." On a well-configured system that's been running for hours this rarely matters, but on a system that was recently rebooted at an unfortunate time, it's a real gap.

Second, **no missed-run recovery**: if the system is powered off at 2:00 AM (scheduled backup time) and powers back on at 3:00 AM, `cron` does not run the missed job. The backup simply doesn't happen for that day, with no record of the miss in any log. This is the behavior `Persistent=true` in a `systemd` timer explicitly corrects.

Third, **execution environment isolation**: `cron` executes jobs with a deliberately minimal environment — no sourced shell initialization, a restricted `PATH`, no terminal. A script that works perfectly when run interactively can fail under `cron` because a dependency lives in a `PATH` entry that only exists in the interactive shell's configuration. Part 4's script was carefully written to use absolute paths precisely to work around this, but it's a sharp edge that catches engineers who aren't aware of it.

What systemd timers provide

A systemd timer is a `.timer` unit that activates a corresponding `.service` unit on a defined schedule. Because both units live within systemd's dependency graph, a timer can express: "don't run this service until `network-online.target` is active," which cron fundamentally cannot. The service unit runs in a clean, predictable execution environment defined by systemd's own unit directives rather than by whatever shell environment happened to exist — more restrictive than even cron's minimal environment, but explicitly documented and consistent. Output goes directly to the journal with no intermediate mail spool, no configuration required. `Persistent=true` in the timer unit means a missed activation (system was off at the scheduled time) is caught and run on the next boot.

The five-field cron syntax (still worth knowing)

Even though this lab's final implementation uses systemd timers, cron remains the dominant scheduling mechanism across Linux infrastructure broadly. The syntax: five space-separated fields — minute (0-59), hour (0-23), day-of-month (1-31), month (1-12), day-of-week (0-7, where both 0 and 7 mean Sunday). A `*` in any field means "every value." A `*/n` means "every n units." `0 2 * * *` means "at 02:00, every day." The most important subtlety: day-of-month and day-of-week are OR'd, not AND'd, when both are restricted simultaneously — a rarely-needed behavior that surprises engineers who encounter it.

The execution environment problem (a production failure worth documenting)

The original plan used a personal passphrase-protected key triggered via cron. This produces two compounding failures: cron's minimal environment means the key's path may not resolve correctly if specified relatively, and cron has no interactive terminal to prompt for the passphrase even if the path is correct. The result is a backup job that appears scheduled, never succeeds, and — if cron's default mail-based output capture is the only visibility mechanism — fails completely silently for however long it goes unchecked. This specific failure mode is what drove the migration to both a passphrase-free M2M key (Part 4) and systemd timers (this part).

5.3 Architecture

A systemd service unit (`edgesecure-backup.service`) defines the backup job — what to run, as whom, and with what dependencies. A systemd timer unit (`edgesecure-backup.timer`) defines when to activate that service. The two units are linked by sharing the same base name; systemd discovers this relationship automatically. The timer is what gets enabled at boot; the service is what actually runs the script. This separation of "when" (timer) from "what" (service) is architecturally cleaner than cron's single-line conflation of schedule and command.

5.4 Linux Internals

systemd timers integrate with the same unit activation mechanism used by every other systemd service on the system. When the timer fires, systemd creates a transient service instance — essentially a fresh, isolated execution environment with `User=root`, no inherited shell state, and stdout/stderr connected to the journal rather than any terminal or mail spool. The `After=network-online.target` directive in the service unit means systemd's dependency resolver will not start the service (even if the timer fires) until the network-online target is active, which on this lab's configuration means WireGuard's `wg-quick@wg0.service` and the underlying physical network interfaces are both confirmed up.

`Persistent=true` in the timer unit interacts with systemd's persistent timestamp tracking: systemd records the last time each timer successfully fired in `/var/lib/systemd/timers/`. On

boot, it checks whether any persistent timer's last-fire timestamp is older than its **OnCalendar** interval — if so, it fires immediately rather than waiting for the next scheduled time. This is the missed-run recovery behavior cron cannot provide.

RandomizedDelaySec=15m adds a random delay up to 15 minutes after the scheduled time before actually firing. This is a production-grade practice for preventing "thundering herd" problems when many systems share the same scheduled time — irrelevant in a three-node lab but worth understanding as a design principle for larger fleets.

5.5 Implementation

Step 1 — Create the service unit

/etc/systemd/system/edgesecure-backup.service:

```
[Unit]
Description=EdgeSecure Hybrid Infrastructure Backup
Documentation=man:rsync(1)
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
User=root
ExecStart=/usr/local/sbin/edgesecure-backup.sh
```

Type=oneshot tells systemd this service runs a script to completion and exits — it is not a long-lived daemon. systemd waits for the script to exit before marking the service run complete, which matters for timer accounting (the next timer activation is calculated from when the current run finished, not when it started).

Step 2 — Create the timer unit

/etc/systemd/system/edgesecure-backup.timer:

```
[Unit]
Description=Daily EdgeSecure Hybrid Infrastructure Backup

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
RandomizedDelaySec=15m

[Install]
WantedBy=timers.target
```

Step 3 — Enable and start the timer

```
sudo systemctl daemon-reload
sudo systemctl enable --now edgesecure-backup.timer
```

Note: enable the **timer**, not the service. The service is activated by the timer; enabling it directly would mean it runs at every boot, which is not the intent.

Step 4 — Confirm the timer is active and next activation is scheduled

```
sudo systemctl list-timers edgesecure-backup.timer
```

5.6 Command Breakdown

<code>systemctl daemon-reload</code>	Reloads systemd's unit file index after new files are created — required before systemd can see newly created units
<code>systemctl enable --now edgesecure-backup.timer</code>	Enables the timer for automatic activation at boot (creates a symlink in <code>timers.target.wants/</code>) and starts it immediately in the current session
<code>systemctl list-timers</code>	Shows all active timers, their last-fire time, next scheduled activation, and the unit they activate — the equivalent of <code>crontab -l</code> for systemd timers
<code>systemctl status edgesecure-backup.service</code>	Shows the outcome of the most recent service run, including exit code and any output captured to the journal
<code>journalctl -u edgesecure-backup.service</code>	Full journal output for every run of the service, including the <code>logger</code> lines from the backup script itself

5.7 Configuration Breakdown

- **After=network-online.target` / `Wants=network-online.target** — **After** means "don't start until this target is active." **Wants** means "try to activate this target if it isn't already, but don't fail if it can't be." Together they express "I need the network, but I won't cause a boot failure if it's unavailable — I just won't run until it's ready."
- **Type=oneshot** — critical for correct timer accounting. Without this, systemd might consider the service "active" as soon as the process starts, potentially triggering the next timer activation before the current run finishes.
- **OnCalendar=*-*-* 02:00:00** — systemd's calendar expression syntax: `year-month-day hour:minute:second`, with `*` as wildcard. More readable than cron's five-field syntax for absolute schedules. `systemd-analyze calendar '*-*-* 02:00:00'` can validate and show the next few activation times for any calendar expression.
- **Persistent=true** — the single most operationally significant difference from cron. Without it, a system that was off at 2:00 AM simply misses the backup. With it, the backup runs on next boot, with a journal record of when it actually fired.
- **WantedBy=timers.target** — timers need to declare themselves as wanted by `timers.target` (the systemd target that activates all enabled timers) in order for `systemctl enable` to install the symlink that makes them start at boot. Without this [Install] section, `systemctl enable` has nowhere to install the symlink.

5.8 Packet Flow

The timer itself generates no network packets — it is purely a local process-scheduling mechanism operating entirely within systemd's internal state machine. The network packet flow that results from a timer-triggered run is identical to the rsync packet flow traced in Part 4, Section 4.8. The timer's contribution to the sequence is only determining when that flow begins.

5.9 Process Flow

1. systemd's timer daemon (`systemd` PID 1 itself, not a separate daemon) tracks the next activation time for all enabled timers.
2. At 2:00 AM (plus up to 15 minutes random delay), the timer unit activates.
3. systemd checks the service unit's dependencies — specifically `After=network-online.target`. If the network isn't up, systemd waits rather than failing immediately.
4. Once dependencies are satisfied, systemd forks and execs `/usr/local/sbin/edgesecure-backup.sh` as root in a clean environment.
5. The script runs both rsync operations sequentially, each writing success or failure to the journal via `logger`.
6. The script exits; systemd records the exit code and run timestamp in the journal and in the persistent timer state file.
7. `journalctl -u edgesecure-backup.service` now shows the complete run record, including every `logger` line from the script, accessible to any engineer without having to check a separate mail spool or log file.

5.10 Verification

```
# Confirm the timer is active and shows a next activation time:
sudo systemctl list-timers edgesecure-backup.timer
```

```
# Manually trigger the service immediately (bypassing the timer schedule):
sudo systemctl start edgesecure-backup.service
```

```
# Check the outcome:
sudo systemctl status edgesecure-backup.service
journalctl -u edgesecure-backup.service --since "5 minutes ago"
```

`systemctl start edgesecure-backup.service` (starting the service directly, not the timer) is the correct way to test the backup job on demand without waiting for the scheduled time — equivalent to running the script manually but within the exact same systemd execution environment the timer would use, making it a faithful test of production behavior rather than an approximation.

5.11 Failure Simulation

Break 1 — Service unit missing `After=network-online.target`.

Symptom: backup runs triggered shortly after boot fail with SSH connection errors — the WireGuard tunnel isn't established yet when the job fires.

Root cause: without the network dependency, systemd starts the service as soon as its own timer fires, regardless of whether the network stack is ready.

Diagnosis: `systemctl list-dependencies edgesecure-backup.service` shows whether `network-online.target` appears in the dependency graph. `journalctl -u edgesecure-backup.service` for a boot-time run shows SSH `ConnectTimeout` errors rather than authentication errors.

Fix: add `After=network-online.target` and `Wants=network-online.target` to the `[Unit]` section and reload: `sudo systemctl daemon-reload`.

Break 2 — Timer enabled but service never runs.

Symptom: `systemctl list-timers` shows the timer as active with a future next-activation time, but after that time passes, no journal entries appear for the service.

Root cause: the service unit has an error that prevents it from starting — a missing `ExecStart` path, a permission error on the script, or a syntax error in the unit file.

Diagnosis: `sudo systemctl start edgesecure-backup.service` immediately reveals the error that the timer would have silently swallowed until the next scheduled check. `journalctl -u edgesecure-backup.service` shows the failure detail.

Fix: correct the unit file error and `sudo systemctl daemon-reload`.

Break 3 — `Persistent=true` not set; missed backup goes undetected.

Symptom: system is rebooted at 1:58 AM; 2:00 AM passes while the system is still booting; backup never runs; no journal entry records the miss; engineer assumes backup ran because "it's scheduled."

Root cause: without `Persistent=true`, systemd makes no attempt to recover missed activations.

Diagnosis: `systemctl list-timers` shows `LAST` as the previous successful run (potentially yesterday), with no indication that today's was missed.

Fix: add `Persistent=true` to the timer's `[Timer]` section, reload, and re-enable. Going forward, a reboot that crosses the scheduled time will trigger the backup on next boot automatically.

5.12 Troubleshooting

Diagnostic order: is the timer itself active (`systemctl list-timers`)? Does manually starting the service succeed (`systemctl start edgesecure-backup.service`)? If not, what specific error appears (`journalctl -u edgesecure-backup.service`)? Is the failure in the systemd layer (unit file, permissions, dependency) or in the script itself (visible once the unit-level issues are resolved)? Does the script succeed when invoked directly as root (`sudo /usr/local/sbin/edgesecure-backup.sh`) confirming the script is correct independent of the systemd execution environment? Each layer is independently testable, and testing them in order prevents misattributing a unit-file syntax error to a network connectivity problem.

5.13 Production Engineering Notes

- **Monitoring timer health:** `systemctl list-timers` shows last and next activation times for all enabled timers — useful as a quick sanity check, but not a substitute for alerting on failed service runs. In production, alerting on `edgesecure-backup.service` entering a failed state (via the same centralized logging pipeline built in Part 2) is the

correct posture, since a timer that fires successfully but whose service exits with an error looks "healthy" from the timer's perspective.

- **Concurrency protection:** systemd's `Type=oneshot` combined with the timer's default behavior prevents concurrent runs — if a service instance is already running when the timer fires again, systemd won't start a second instance. This is simpler and more reliable than cron's manual `flock` approach.
- **cron is not obsolete:** for simple, stateless, time-based jobs on systems where systemd is not the init system (containers, some embedded environments) or where a team's operational familiarity is firmly with cron, cron remains entirely appropriate. The choice between them should be driven by the specific workload's requirements — this workload's network dependency and missed-run recovery requirement are what tip the balance toward systemd timers here.

5.14 Interview Reference

Q: What is the specific advantage of systemd timers over cron for this backup workload?

A: Two concrete advantages that cron cannot provide: `After=network-online.target` ensures the backup job only runs once the WireGuard tunnel is confirmed up rather than firing blindly at the scheduled time regardless of network state, and `Persistent=true` ensures a missed activation (system was off at 2:00 AM) runs on next boot rather than silently not running at all. For stateless, network-independent jobs, cron's universality often makes it the better choice. This job has a hard network dependency and a missed-run recovery requirement, which tip the balance.

Q: A systemd timer shows as active with a future next-activation time, but the service never runs when that time arrives. What do you check first?

A: Manually start the service directly with `systemctl start edgesecure-backup.service` — this immediately surfaces any error the timer would have swallowed until the next diagnostic cycle. Unit file errors, script permission errors, and dependency failures all manifest immediately this way, rather than requiring the engineer to wait for the next scheduled activation to observe the failure.

Q: Why `Type=oneshot` for this service unit specifically?

A: The backup script runs to completion and exits — it is not a daemon. `Type=oneshot` tells systemd to wait for the process to fully exit before marking the service run complete. This matters for timer accounting (the next activation is calculated from run-end, not run-start), for accurate journal timestamps, and for correct dependency behavior if anything else in the unit graph were to `After=edgesecure-backup.service`.

Failure Lab Index — Cross-Technology Enterprise Troubleshooting Scenarios

This section consolidates the failure simulations from each part into a single quick-reference index, organized by the symptom an engineer would actually observe first, since real incidents rarely announce which subsystem is responsible.

<code>wg-quick</code> fails: "Key is not correct length or format" at a file path	Key path pasted instead of key content	1.11
WireGuard: <code>wg show</code> never shows a handshake, 0 B received, KiB sent	Azure NSG blocking inbound UDP 51820	1.11, Break 2
WireGuard: handshake succeeds, ICMP works, TCP apps fail	<code>wg0</code> in wrong firewall zone — port 514 not allowed	1.11, Break 3
rsyslog "cannot connect to 10.100.0.1:514" repeated every 30s	Firewall zone missing <code>514/tcp</code> explicit rule	2.11, Break 1
rsyslog forwarding configured but server receives nothing	Shell command accidentally pasted into <code>.conf</code> file	2.11
<code>/var/log/remote/</code> empty despite confirmed TCP packet delivery	SELinux AVC denial on write path	2.11, Break 2
Expected log file not found (e.g. <code>logger.log</code> missing)	<code>%PROGRAMNAME%</code> tag differs from assumed value	2.4
<code>backend01</code> cannot ping <code>edge01</code> but can forward logs to it	Different firewall rules for ICMP vs TCP 514	2.14
Fail2Ban shows "Currently banned: N" but SSH still connects	Wrong <code>banaction</code> for distribution — <code>firewallcmd-rich-rules</code> on Ubuntu	3.11, Break 2
Fail2Ban shows zero detections during an active attack	Wrong <code>logpath</code> or <code>backend</code> for the distribution	3.11, Break 1
Backup script hangs indefinitely with no output	Passphrase-protected key used in automation	4.11, Break 1
rsync exits 255 on every automated run	Automation key not in remote <code>authorized_keys</code>	4.11, Break 2

Spurious FAILURE alerts on every backup run	Exit code 24 treated as error instead of harmless warning	4.6
Service fires but fails on backend02 pull at boot	WireGuard not established before network-online.target	5.11, Break 1
Unit file changes not taking effect after edit	systemctl daemon-reload not run after unit file modification	5.11, Break 2
Timer shows active but service never ran when expected	Service unit has syntax error — use systemctl start to surface it	5.11, Break 2

Engineering Outcome

EdgeSecure's infrastructure now has an actual operational backbone where it previously had isolated capability. Every design decision documented in this phase was driven by a real constraint encountered during implementation — not by convention or preference.

The WireGuard overlay corrects a real architectural defect: the original assumption that Azure could reach `192.168.56.10` was not a configuration gap but a structural impossibility proven at the packet level in Section 2. The role assignment — `backend02` listens, `edge01` initiates — was determined by the topology itself, not by any template. The Azure NSG, silent at the OS level, was the actual blocker confirmed by zero bytes received on an otherwise healthy-looking `wg show` output.

Centralized rsyslog logging exposed multiple distinct failure modes across the troubleshooting process: a shell command accidentally pasted into a config file, a firewall zone with a missing port rule (confirmed by SYN-with-no-SYN-ACK in tcpdump), and the ICMP-vs-TCP independence lesson that a host unreachable by ping can simultaneously accept log forwarding on a different protocol.

Fail2Ban required distribution-awareness that the original manual did not capture: `banaction = firewallcmd-rich-rules` is wrong on Ubuntu — it references a tool that does not exist and fails silently, leaving Fail2Ban's internal state showing bans that are not enforced at the kernel level. `banaction = ufw` is the correct value. Real external attackers were already in the banned list before any deliberate test was run on `backend02`, confirming the jail was working in genuine production conditions.

The rsync disaster recovery design was materially hardened from the original plan: a dedicated passphrase-free M2M automation key (`/root/.ssh/id_ed25519_automation`) replaced the personal interactive key; `--rsync-path="sudo rsync"` resolved Permission Denied errors on privileged files; cloud-native key distribution bypassed Azure's `ssh-copy-id` incompatibility; and exit code 24 handling prevented spurious failure alerts from transient lock files on live systems.

A systemd timer replaced cron as the scheduling mechanism, specifically for the network-dependency awareness (`After=network-online.target`) and missed-run recovery (`Persistent=true`) that cron cannot provide — both directly relevant to an environment where the WireGuard tunnel must be established before the backup job can reach `backend02`.

Final Engineering State

- **Network:** `backend02` listening on UDP 51820, `edge01` initiating with `PersistentKeepalive = 25`. `wg0` assigned to `internal` firewall zone on `edge01` with explicit port grants. Kernel IP forwarding active. `backend02`'s public IP and all existing Phase 4 traffic entirely undisturbed.
- **Logging:** rsyslog server on `edge01` listening on TCP 514 with `514/tcp` explicitly opened in the `internal` zone. Dynamic per-host templating filing to `/var/log/remote/<hostname>/<programname>.log`. `backend01` forwarding over Host-only (`192.168.56.10`), `backend02` forwarding over WireGuard overlay (`10.100.0.1`). Disk-assisted queueing active on both clients.
- **Intrusion Response:** Fail2Ban active on both backend nodes with `banaction = ufw` (correct for Ubuntu). SSH jails tuned with `backend01`-specific test approach (private network, tested from `edge01`). Real external attack traffic confirmed banned on `backend02`.
- **Disaster Recovery:** Dedicated root-owned M2M automation key at `/root/.ssh/id_ed25519_automation`. Pull-based rsync from `edge01` with `--rsync-path="sudo rsync"` for privileged file access. Exit code 24 handled as non-failure. Backup directories under `/backup/` owned by `eudofia:eudofia` for unprivileged access.
- **Automation:** systemd timer (`edgesecure-backup.timer`) triggering nightly at 02:00 with `After=network-online.target` and `Persistent=true`. Outcome flowing into journal and rsyslog centralized pipeline via `logger`.

Operational Summary

An engineer operating this infrastructure can now trust that: a compromised or destroyed node leaves its audit history intact elsewhere; a brute-force attempt against either backend node is automatically and swiftly contained at the kernel level without manual intervention; a catastrophic data-loss event on either backend node has a current, verified recovery point no more than 24 hours old; and the cross-cloud network boundary that made all of this previously impossible for `backend02` specifically has been correctly diagnosed and durably resolved at the architectural level rather than worked around with a fragile improvisation.

Key Engineering Lessons

1. **Diagnose the actual structural cause before reaching for a tool.** The original plan's failure was not a misconfiguration — it was a wrong assumption about address space reachability. Understanding exactly where and why a packet dies (Section 2's RFC 1918 walk-through) is what correctly scoped the fix. NAT rules and firewall changes would not have solved it.
2. **Role assignment in a VPN must follow network topology.** `backend02` listens because it has a publicly reachable IP. `edge01` initiates because it is behind NAT. These are not preferences — they are constraints. A configuration that reverses them looks syntactically valid but produces a tunnel that never establishes a handshake.
3. **Detection, decision, and enforcement are independently failable layers.** Fail2Ban's filter, jail policy, and firewall integration can each fail without the others. A ban recorded in Fail2Ban's internal state without a corresponding kernel-level rule — caused by `firewallcmd-rich-rules` on Ubuntu — is not a ban. Verify enforcement independently with `ufw status numbered`, not just with `fail2ban-client status`.
4. **Unattended automation has different constraints than interactive use.** Passphrase-protected keys, sudo password prompts, shell initialization variables, and relative paths all work interactively and fail silently in automation. The M2M key, `--rsync-path="sudo rsync"`, `BatchMode=yes`, and absolute paths in the backup script each address one specific failure mode of this class.
5. **Reachability is per-protocol, per-port — never binary.** `backend01` cannot ping `edge01` but forwards logs to it successfully. Both behaviors are correct simultaneously. ICMP and TCP 514 are evaluated by completely separate firewall rules. Production monitoring never uses ping as a definitive reachability test.